

**Kutaisi International University
School of Computer Science
Bachelor's Program in Computer Science**

Mariam Katamashvili

**Formalization of Crisp and Fuzzy Derivatives of Regular Expressions in
Rocq**

The thesis is completed for obtaining the academic degree of
Bachelor of Computer Science

Besik Dundua

Professor

Furio Honsell

Professor

Kutaisi, 2026

Declaration

I, the author of this bachelor's thesis, declare that the thesis is my own original work and does not contain any material previously published, submitted for publication, or presented for defense by other authors, unless properly referenced or cited in accordance with academic standards.

Mariam Katamashvili

June 29, 2026

Abstract

This thesis studies Brzowski derivatives of regular expressions in two settings, the classical crisp one and a fuzzy one based on similarity, and verifies the theorems in the Rocq proof assistant. In the crisp setting it defines derivatives of regular languages and expressions, the correspondence between the syntactic and the semantic derivative, the finiteness of the set of derivatives, and the construction of the deterministic automaton that has canonical derivatives as its states. The original contribution is the fuzzy layer, which replaces exact symbol matching by a similarity relation cut at a threshold μ . Its main observation is that, under the minimum triangular norm, cut-similarity is an equivalence relation. A fuzzy derivative over the alphabet is then just an ordinary derivative over the quotient alphabet obtained by identifying similar symbols. Through this quotient construction regularity, the semantic-syntactic correspondence, finiteness, and correctness of the fuzzy automaton all carry over from the crisp theory. The automaton recognises exactly the words that lie within threshold μ of some word of the original language. The claims are checked by a proof assistant rather than argued only on paper.

Keywords: regular expressions, Brzowski derivatives, finite automata, fuzzy similarity, triangular norms, formal verification, Rocq.

Contents

Declaration	i
Abstract	ii
1 Introduction	1
2 Literature Review	2
3 Methodology and Results	3
3.1 Preliminaries	5
3.2 Derivatives of Regular Languages and Regular Expressions	18
3.3 Finite Automata	22
3.4 Fuzzy Proximity and Similarity Relations	28
3.5 Fuzzy Derivatives and Fuzzy Automata	37
4 Conclusion	42
Bibliography	43
Appendices	44

1 Introduction

Regular expressions were introduced by Stephen Kleene in the 1950s [11] as a notation for describing the patterns that a finite-state machine can recognise. The idea was to capture, with a few simple operators for union, concatenation and Kleene star, exactly the languages accepted by finite automata. That correspondence between a piece of syntax and a machine has proved to be one of the most durable ideas in computer science. Regular expressions are interesting precisely because they sit at this meeting point of theory and practice. On the theoretical side they describe a well-understood class of languages and come with a rich algebra. On the practical side they are everywhere, from the lexical analysers inside every compiler, through search and text processing, to validation of input and scanning of network traffic. A notation that a programmer types in seconds is backed by decades of automata theory, and that is what makes getting its foundations right worthwhile.

The common way to run a regular expression is to compile it into a finite automaton and then let the machine consume the input, using one of the classical constructions of Thompson [24], Glushkov [8], or McNaughton and Yamada [15]. There is also an older and more direct idea, due to Brzozowski [3], that this thesis takes as its starting point. Rather than building a separate machine, one can differentiate the expression itself: the derivative of an expression with respect to a symbol describes what remains to be matched after that symbol has been consumed, so iterating it across an input word is already a way of running the expression. When derivatives are kept in a canonical form there are only finitely many of them, and taking those canonical derivatives as states yields a deterministic automaton with no auxiliary construction at all. The approach has aged well and was reexamined from a modern implementation perspective by Owens, Reppy and Turon [17].

Exact matching, however, is often stricter than an application really wants. Text is noisy, symbols are mistyped or substituted, and two inputs that a person would treat as the same can differ in a handful of places. This is why fuzziness is interesting: a great deal of real matching is approximate, and a theory that insists on equality of symbols cannot describe it. A natural way to relax the rigidity is to replace equality between symbols by a notion of similarity, so that two symbols match not only when they are identical but also when they are close enough. Closeness is measured by a value in $[0, 1]$ and combined using triangular norms of fuzzy logic [16]. The same idea appears in similarity-based logic programming [19], where unification is replaced by matching up to a similarity relation. This thesis asks what becomes of Brzozowski's derivative theory once symbols match by similarity. If the consumed symbol only has to be similar to the first symbol of a word, up to a threshold μ , does the derivative-based theory of regular languages and automata still hold together?

It does, for a simple reason. When similarity is combined with the minimum triangular norm, cutting the similarity relation at a threshold μ gives an equivalence relation. A fuzzy derivative over the alphabet is then just an ordinary derivative over the quotient alphabet, where symbols similar within

μ are identified. The crisp theory transfers to the fuzzy setting through this quotient construction. Regularity, the correspondence between the syntactic and the semantic derivative, finiteness of the set of derivatives, and correctness of the derivative automaton all carry over. The fuzzy automaton accepts exactly the words that lie within tolerance μ of some word of the language.

Derivative and finiteness arguments are easy to get subtly wrong. A proof assistant checks every step mechanically. The crisp theory is ergo mechanised in the Rocq proof assistant [23], building on the Coquand–Siles formalisation of regular-expression equivalence via Brzozowski derivatives [6], [22]. It covers regular-expression syntax and semantics, nullability, canonicalisation, the derivative correspondence and automaton correctness, and the similarity infrastructure on which the fuzzy layer rests. The full development is available in a repository [10], with the code presented next to the mathematics it verifies.

The rest of the thesis is organised as follows. Section 2 reviews the existing literature on regular expressions, derivatives, their formalisation, and fuzzy automata. Section 3 contains the technical development in two layers: the crisp theory of derivatives and the derivative automaton, presented together with its Rocq formalisation, followed by the fuzzy layer with its definitions, the quotient construction, and the fuzzy automaton. Section 4 summarises the results.

2 Literature Review

The theory this thesis builds on splits into a few strands that have largely developed in parallel. It is worth seeing where each of them stands before combining them. The first strand is the classical correspondence between regular expressions and finite automata. The standard constructions that realise an expression as a machine are Thompson’s translation into a nondeterministic automaton with epsilon transitions [24], the position automaton of Glushkov [8], and the state-graph method of McNaughton and Yamada [15]. These remain the textbook routes from syntax to machine, and they treat the automaton as an object to be built alongside the expression rather than read off from it.

A second strand reasons about regular expressions algebraically. Conway’s account of regular algebra and finite machines [4] organises the equational theory, while Salomaa gave complete axiom systems for the algebra of regular events [18] and Kozen later established completeness for Kleene algebra and the algebra of regular events [12]. This line is concerned with when two expressions denote the same language and with axiomatising that equivalence. That is a different question from how to match efficiently, but a closely related one, since deciding equivalence is one of the things a good normal form for expressions should help with.

The derivative approach sits between these two and is the one this thesis takes as its starting point. Brzozowski introduced derivatives of regular expressions [3]. He showed that differentiating an expression with respect to a symbol, together with a test for whether the empty word is accepted, is

enough both to decide membership and to construct a deterministic automaton, provided expressions are identified up to a few simplifications so that only finitely many derivatives arise. Owens, Reppy and Turon re-examined the construction from a modern implementation perspective [17]. They showed that it remains both elegant and practical, particularly because it extends smoothly to intersection and complement. Derivatives are attractive precisely because the automaton is latent in the expression rather than assembled separately, which is also what makes the method a good fit for mechanised reasoning.

That fit has been borne out by work on formalising regular languages in proof assistants. The development this thesis extends descends from the Coquand–Siles decision procedure for regular-expression equivalence in type theory [6] and its accompanying Rocq formalisation of Brzozowski derivatives [22], carried out in the Rocq proof assistant [23]. Mechanising this material forces every simplification and every finiteness argument to be made explicit, which is exactly where paper proofs tend to be informal. It is also the foundation on which the similarity layer of the present work is added.

The fuzzy side of the picture has its own substantial literature, though it has mostly grown from a different root. Most of this work gives automata and expressions membership values rather than crisp acceptance, so a word is recognised to a degree. Li and Pedrycz studied fuzzy finite automata and fuzzy regular expressions with membership values in lattice-ordered monoids [13], Stamenković and Ćirić gave constructions of fuzzy automata from fuzzy regular expressions [21], and Garhwal, Jiwari and Tomasiello surveyed fuzzy regular languages and fuzzy automata more broadly [7]. A related quantitative and coalgebraic perspective appears in the quantitative Kleene coalgebras of Silva, Bonchi, Bonsangue and Rutten [20] and, more recently, in the extension of the quantitative pattern-matching paradigm by Alves, Kesner and Ramos [1]. The technical background for all of this is the theory of triangular norms and fuzzy logic, of which Metcalfe’s notes give a brief account [16].

What is distinctive about the route taken here is the source of the fuzziness. Rather than attaching weights to transitions or to acceptance, this thesis keeps acceptance crisp and instead loosens the matching of symbols. A symbol then matches any other symbol that is similar to it within a chosen threshold. This is the same move that Sessa made in approximate logic programming, where unification is replaced by matching up to a similarity relation [19], transposed from resolution to derivatives. The payoff is that, under the minimum T-norm, the fuzzy theory reduces to the crisp one over a quotient alphabet, which keeps the derivative machinery and its formalisation intact instead of requiring a separate weighted theory. The present work therefore connects the derivative-based, formally verified theory of regular expressions with similarity-based approximate matching.

3 Methodology and Results

Rocq (until recently called Coq) is an interactive proof assistant built on a constructive dependent type theory, the Calculus of Inductive Constructions [2], [5], a descendant of Martin-Löf’s intuitionistic

type theory [14]. What makes such a system able to check mathematics is the *propositions-as-types* principle, also known as the Curry–Howard correspondence [9]. A proposition is read as a type, and a proof of that proposition is read as a *term* (a program) whose type is exactly that proposition. Proving a statement therefore means constructing an object of the corresponding type, and a statement is regarded as true precisely when its type is inhabited.

Under this reading the logical connectives become the type-forming operations of a functional programming language, so that proofs are literally functional programs. A proof of an implication $A \Rightarrow B$ is a function that turns any proof of A into a proof of B ; a proof of a conjunction $A \wedge B$ is a pair holding a proof of each conjunct; a proof of a disjunction $A \vee B$ is a tagged value that records which side holds together with a proof of it; a proof of a universal statement $\forall x. P(x)$ is a (dependent) function mapping each x to a proof of $P(x)$; and a proof of an existential statement $\exists x. P(x)$ is a (dependent) pair consisting of a witness a together with a proof of $P(a)$. This is the *proofs-as-programs* paradigm. Because proofs are programs, they can be run, and the computation steps of the type theory correspond to the simplification of proofs.

Both paradigms depend on the logic being *constructive* (intuitionistic) rather than classical. In intuitionistic logic a proof of $\exists x. P(x)$ must exhibit a witness, and a proof of $A \vee B$ must say which disjunct holds. This is what makes the corresponding proof terms carry the data a program needs in order to compute, namely the witness and the chosen branch. Classical logic abandons that discipline. Its principles, the law of excluded middle $A \vee \neg A$ and double-negation elimination $\neg\neg A \Rightarrow A$, are not theorems of intuitionistic logic. A classical proof of an existential need not produce any witness. One may prove that an object exists by deriving a contradiction from its non-existence, without ever naming it. Such a proof has no computational content, and there is no program to extract from it. Were Rocq founded on classical rather than intuitionistic logic, the propositions-as-types reading would survive only as a formal analogy. Proofs would no longer be programs that can be run. The witnesses and decision procedures this thesis relies on could not be obtained from the proofs that justify them, and the executable character of the formalisation would be lost. The constructive foundation is ergo not an incidental choice. It is what makes the formalised theory compute. (Classical axioms can be added to Rocq consistently when needed, but doing so reintroduces this loss of computational content, and the development does not use them.)

In practice one rarely writes these proof terms by hand, because for any nontrivial statement the term is large. Instead Rocq is driven by *tactics*, commands that build the proof term indirectly, working backwards from the goal. At each point the system displays a *proof state* consisting of the hypotheses available and the goal still to be proved. Each tactic transforms that state, reducing the goal or splitting it into subgoals while assembling the corresponding fragment of the proof term. The tactics mirror the type-theoretic operations described above. For instance `intro` moves the premise of an implication into the hypotheses (introducing a function abstraction), `apply` uses an implication or lemma to reduce the

goal to its premises (function application, *i.e.* modus ponens), and `induction` invokes the induction principle of an inductive datatype. When every subgoal has been discharged, the command `Qed` submits the finished term to Rocq’s kernel, which type-checks it independently. The tactics are only a way of producing the term. It is the kernel’s check, not the tactic script, that is believed. The Rocq listings shown in this section are the statements and definitions from this process. The tactic scripts that prove them are available in the repository [10].

3.1 Preliminaries

Definition 3.1 (Alphabet, words, and languages).

- *An alphabet Σ is a finite, nonempty set of symbols.*
- *A word over Σ is a finite sequence of symbols from Σ .*
- *The empty word is denoted by ε .*
- *The set of all words over Σ is Σ^* .*
- *A language over Σ is a subset $\mathcal{L} \subseteq \Sigma^*$.*

In Rocq the alphabet is abstract, given by the module type `SYM`, which fixes a type `A` of symbols with decidable equality (an `eqType`). A word is then a finite list of symbols, `seq A`, with the empty word written `[::]`, and a language is a boolean predicate on words, `pred word`, so that membership is a decidable test.

Listing 1: Alphabet, words, and languages (`Alphabet.v`, `Languages.v`).

```

1 (* A plain alphabet of symbols. This is the minimal interface needed for
2    words, languages, regular-expression syntax, and semantics. *)
3 Module Type SYM.
4   Parameter A : eqType.
5 End SYM.
6
7 (* A word over the alphabet A is a finite sequence of symbols from A.
8    The empty word  $\varepsilon$  is written directly as [::]. *)
9 Definition word := seq A.
10
11 (* A language over A is a decidable predicate on words. *)
12 Definition language := pred word.
```

Definition 3.2 (Regular expressions). *The regular expressions, or regular events, over an alphabet Σ , written E^Σ , are defined by the grammar*

$$r ::= \emptyset \mid \varepsilon \mid a \ (a \in \Sigma) \mid r + r \mid r \cdot r \mid r^* \mid r \& r \mid \neg r,$$

where a ranges over Σ . The intended meanings are: $\emptyset$ the empty language; ε the language $\{\varepsilon\}$; a the singleton $\{a\}$; $+$ union; $\cdot$ concatenation; * Kleene star; $\&$ intersection; and $\neg$ complement.

Definition 3.3 (Restricted regular expressions). *The restricted regular expressions over Σ , written R^Σ , are those regular expressions in which the constructors $\&$ and $\neg$ do not occur; that is, they are built from $\emptyset$, ε and the symbols $a \in \Sigma$ using only $+$, $\cdot$ and * .*

Definition 3.4 (Semantic interpretation). *For a regular expression r over Σ , its semantic interpretation is the language $\mathcal{L}(r) \subseteq \Sigma^*$ defined inductively by*

$$\begin{aligned} \mathcal{L}(\emptyset) &= \emptyset, & \mathcal{L}(\varepsilon) &= \{\varepsilon\}, \\ \mathcal{L}(a) &= \{a\}, \ a \in \Sigma, & \mathcal{L}(r + s) &= \mathcal{L}(r) \cup \mathcal{L}(s), \\ \mathcal{L}(r \cdot s) &= \mathcal{L}(r) \cdot \mathcal{L}(s), & \mathcal{L}(r^*) &= \bigcup_{n \geq 0} \mathcal{L}(r)^n, \\ \mathcal{L}(r \& s) &= \mathcal{L}(r) \cap \mathcal{L}(s), & \mathcal{L}(\neg r) &= \Sigma^* \setminus \mathcal{L}(r). \end{aligned}$$

Example. Over $\Sigma = \{a, b\}$ the expression $(a + b)^* \cdot a$ denotes the language of all words that end in a , since $(a + b)^*$ matches any prefix and the trailing a fixes the last symbol. It is built only from $+$, $\cdot$ and * and so belongs to the restricted fragment R^Σ . By contrast $\neg(a \cdot b)$ denotes every word except ab , and $a \& (a + b)$ denotes $\{a\}$; both use $\neg$ or $\&$ and are ergo not restricted, though still regular.

In Rocq, regular expressions are an inductive datatype whose eight constructors correspond one-to-one to $\emptyset, \varepsilon, a, +, \cdot, ^*, \&, \neg$. The restricted fragment R^Σ of Definition 3.3 is carved out by an inductive predicate `restricted_regex`, which holds of an expression exactly when it is built without `And` and `Neg`. The semantic interpretation $\mathcal{L}(\cdot)$ is the function `den`, defined by a structural recursion that maps each constructor to the matching language operation (`void`, `eps`, `atom`, `plus`, `conc`, `star`, `prod`, `compl`).

Listing 2: Regular expressions, the restricted fragment, and the semantics (`RegexSemantics.v`).

```

1 (* Syntax side of semantic interpretation: the regular expressions over A. *)
2 Inductive regex : Type :=
3   | Empty : regex
4   | Eps   : regex
5   | Char  : A -> regex
6   | Alt   : regex -> regex -> regex
7   | Seq   : regex -> regex -> regex

```

```

8 | Star : regex -> regex
9 | And : regex -> regex -> regex
10 | Neg : regex -> regex.
11
12 (* Predicate checking that a regex is restricted, i.e. contains no And or Neg
   constructors. *)
13 Inductive restricted_regex : regex -> Prop :=
14 | RestrictedEmpty : restricted_regex Empty
15 | RestrictedEps : restricted_regex Eps
16 | RestrictedChar : forall a, restricted_regex (Char a)
17 | RestrictedAlt : forall r s,
18   restricted_regex r -> restricted_regex s -> restricted_regex (Alt r s)
19 | RestrictedSeq : forall r s,
20   restricted_regex r -> restricted_regex s -> restricted_regex (Seq r s)
21 | RestrictedStar : forall r,
22   restricted_regex r -> restricted_regex (Star r).
23
24 (* Semantic interpretation: maps each regex to the language it denotes. *)
25 Fixpoint den (r : regex) : pred word :=
26   match r with
27   | Empty => void
28   | Eps => eps
29   | Char a => atom a
30   | Alt r1 r2 => plus (den r1) (den r2)
31   | Seq r1 r2 => conc (den r1) (den r2)
32   | Star r1 => star (den r1)
33   | And r1 r2 => prod (den r1) (den r2)
34   | Neg r1 => compl (den r1)
35   end.

```

The language operations that den maps onto are defined in `Languages.v`. The empty language, the language of the empty word and the singleton are immediate. Union, intersection and complement are the pointwise boolean connectives. Concatenation and Kleene star are decided by a bounded search over the ways of splitting a word, using helper functions for splitting and for testing every block of a split.

Listing 3: The language operations (`Languages.v`).

```

1 Definition void : language := pred0.
2 Definition eps : language := fun w => w == [::].
3 Definition atom (a : A) : language := fun w => w == [:: a].
4
5 Definition plus (L K : language) : language := fun w => L w || K w.

```

```

6 Definition conc (L K : language) : language :=
7   fun w => has (fun i =>
8     let u := take i w in
9     let v := drop i w in
10    L u && K v) (iota 0 (size w).+1).
11
12 (* Helpers for Kleene star. *)
13
14 (* Checks that every word in sequence belongs to the Language.
15    This is needed to express membership in the Kleene star. *)
16 Fixpoint all_in (L : language) (xs : seq word) : bool :=
17   match xs with
18   | [::] => true
19   | x :: xs' => L x && all_in L xs'
20   end.
21
22 (* Enumerates all finite decompositions of a word into blocks.
23    This is the executable counterpart of the union-of-powers view of L*. *)
24 Fixpoint splits (w : word) : seq (seq word) :=
25   match w with
26   | [::] => [:: [::] ]
27   | a :: w' =>
28     flatten [seq
29       match ds with
30       | [::] => [:: [:: [:: a] ] ]
31       | x :: xs => [:: [:: a] :: ds; (a :: x) :: xs]
32       end
33     | ds <- splits w']
34   end.
35
36 Definition star (L : language) : language := fun w => has (all_in L) (splits w).
37 Definition prod (L K : language) : language := fun w => L w && K w.
38 Definition compl (L : language) : language := fun w => ~~ L w.

```

In many constructions it is useful to know whether the language of an expression contains the empty word.

Definition 3.5 (Nullability). *A regular expression r is nullable if $\varepsilon \in \mathcal{L}(r)$. The nullable function ν is defined by*

$$\nu(r) = \begin{cases} \varepsilon & \text{if } r \text{ is nullable,} \\ \emptyset & \text{otherwise.} \end{cases}$$

Equivalently, ν is characterised by the following equations:

$$\begin{aligned}
\nu(\emptyset) &= \emptyset, & \nu(\varepsilon) &= \varepsilon, \\
\nu(a) &= \emptyset, \ a \in \Sigma, & \nu(r + s) &= \nu(r) + \nu(s), \\
\nu(r \cdot s) &= \nu(r) \& \nu(s), & \nu(r^*) &= \varepsilon, \\
\nu(r \& s) &= \nu(r) \& \nu(s), & \nu(\neg r) &= \varepsilon \text{ if } \nu(r) = \emptyset, \text{ and } \emptyset \text{ if } \nu(r) = \varepsilon.
\end{aligned}$$

Example. The expression a^* is nullable, since $\varepsilon \in \mathcal{L}(a^*)$, so $\nu(a^*) = \varepsilon$. The expression $a \cdot a^*$ is not nullable, since every word it denotes has at least one a , so $\nu(a \cdot a^*) = \emptyset$. For a union the rule gives $\nu(a + \varepsilon) = \nu(a) + \nu(\varepsilon) = \emptyset + \varepsilon = \varepsilon$.

In the formalisation, nullability is the boolean function `has_eps` and ν is `nu`. The lemma `nullable_correct` proves, by induction on the expression, that `has_eps` agrees with semantic membership of the empty word `[ : ]` in the language.

Listing 4: Nullability and its correctness (`Nullable.v`).

```

1 (* Boolean test for whether ε belongs to the language of a regex. *)
2 Fixpoint has_eps (r : regex) : bool :=
3   match r with
4   | Empty      => false
5   | Eps        => true
6   | Char _     => false
7   | Alt r1 r2  => has_eps r1 || has_eps r2
8   | Seq r1 r2  => has_eps r1 && has_eps r2
9   | Star _     => true
10  | And r1 r2   => has_eps r1 && has_eps r2
11  | Neg r1      => ~~ has_eps r1
12  end.
13
14 Definition nu (r : regex) : regex :=
15   if has_eps r then Eps else Empty.
16
17 (* The boolean test has_eps r agrees with the semantic definition of nullable [ : ] \
   in r. *)
18 Lemma nullable_correct : forall r, has_eps r = ([ : ] \in r).
19 Proof.
20   elim=> /=.
21   - by [].
22   - by [].
23   - by move=> a.
24   - move=> r1 IH1 r2 IH2; by rewrite IH1 IH2.
25   - move=> r1 IH1 r2 IH2.

```

```

26     rewrite IH1 IH2.
27     change (([::] \in r1) && ([::] \in r2) =
28         conc (den r1) (den r2) [::]).
29     by rewrite /conc (Languages.conc_nil (A:=A) (den r1) (den r2)).
30 - move=> r IH.
31     have H : star (den r) [::] := Languages.star_nil (A:=A) (den r).
32     by case: (star (den r) [::]) H.
33 - move=> r1 IH1 r2 IH2; by rewrite IH1 IH2.
34 - move=> r IH; by rewrite IH.
35 Qed.

```

The script above and the term below are the same proof in two forms. One is the high level script we write, the other is the low level term Rocq stores and checks. The term makes every induction case and rewrite explicit.

Listing 5: Proof term produced by the script for nullable_correct.

```

1 N.nullable_correct =
2 fun _top_assumption_ : N.RS.regex =>
3 (fun
4   (_evar_0_ : (fun r : N.RS.regex => N.has_eps r = ssrbool.in_mem nil (ssrbool.mem r
5   ))
6     N.RS.Empty)
7   (_evar_0_0 : (fun r : N.RS.regex => N.has_eps r = ssrbool.in_mem nil (ssrbool.mem
8   r))
9     N.RS.Eps)
10  (_evar_0_1 : forall s : eqtype.Equality.sort BoolSym.A,
11    (fun r : N.RS.regex => N.has_eps r = ssrbool.in_mem nil (ssrbool.mem
12    r))
13      (N.RS.Char s))
14  (_evar_0_2 : forall r : N.RS.regex,
15    (fun r0 : N.RS.regex =>
16      N.has_eps r0 = ssrbool.in_mem nil (ssrbool.mem r0)) r ->
17    forall r0 : N.RS.regex,
18    (fun r1 : N.RS.regex =>
19      N.has_eps r1 = ssrbool.in_mem nil (ssrbool.mem r1)) r0 ->
20    (fun r1 : N.RS.regex =>
21      N.has_eps r1 = ssrbool.in_mem nil (ssrbool.mem r1))
22      (N.RS.Alt r r0))
23  (_evar_0_3 : forall r : N.RS.regex,
24    (fun r0 : N.RS.regex =>
25      N.has_eps r0 = ssrbool.in_mem nil (ssrbool.mem r0)) r ->
26    forall r0 : N.RS.regex,

```

```

24         (fun r1 : N.RS.regex =>
25           N.has_eps r1 = ssrbool.in_mem nil (ssrbool.mem r1)) r0 ->
26         (fun r1 : N.RS.regex =>
27           N.has_eps r1 = ssrbool.in_mem nil (ssrbool.mem r1))
28           (N.RS.Seq r r0))
29   (_evar_0_4 : forall r : N.RS.regex,
30     (fun r0 : N.RS.regex =>
31       N.has_eps r0 = ssrbool.in_mem nil (ssrbool.mem r0)) r ->
32     (fun r0 : N.RS.regex =>
33       N.has_eps r0 = ssrbool.in_mem nil (ssrbool.mem r0))
34       (N.RS.Star r))
35   (_evar_0_5 : forall r : N.RS.regex,
36     (fun r0 : N.RS.regex =>
37       N.has_eps r0 = ssrbool.in_mem nil (ssrbool.mem r0)) r ->
38     forall r0 : N.RS.regex,
39     (fun r1 : N.RS.regex =>
40       N.has_eps r1 = ssrbool.in_mem nil (ssrbool.mem r1)) r0 ->
41     (fun r1 : N.RS.regex =>
42       N.has_eps r1 = ssrbool.in_mem nil (ssrbool.mem r1))
43       (N.RS.And r r0))
44   (_evar_0_6 : forall r : N.RS.regex,
45     (fun r0 : N.RS.regex =>
46       N.has_eps r0 = ssrbool.in_mem nil (ssrbool.mem r0)) r ->
47     (fun r0 : N.RS.regex =>
48       N.has_eps r0 = ssrbool.in_mem nil (ssrbool.mem r0))
49       (N.RS.Neg r)) =>
50   N.RS.regex_ind
51   (P:=fun r : N.RS.regex => N.has_eps r = ssrbool.in_mem nil (ssrbool.mem r))
52   _evar_0_
53   _evar_0_0 _evar_0_1 _evar_0_2 _evar_0_3 _evar_0_4 _evar_0_5 _evar_0_6
54   _top_assumption_)
55   eq_refl eq_refl (fun a : eqtype.Equality.sort BoolSym.A => eq_refl)
56   (fun (r1 : N.RS.regex) (IH1 : N.has_eps r1 = ssrbool.in_mem nil (ssrbool.mem r1))
57     (r2 : N.RS.regex) (IH2 : N.has_eps r2 = ssrbool.in_mem nil (ssrbool.mem r2)) =>
58     (fun
59       _evar_0_ : (ssrbool.in_mem nil (ssrbool.mem r1) || N.has_eps r2)%bool =
60       ssrbool.in_mem nil (ssrbool.mem (N.RS.Alt r1 r2)) =>
61       eq_ind_r
62       (fun _pattern_value_ : bool =>
63         (_pattern_value_ || N.has_eps r2)%bool =
64         ssrbool.in_mem nil (ssrbool.mem (N.RS.Alt r1 r2)))
65       _evar_0_ IH1)
66     ((fun
67       _evar_0_ : (ssrbool.in_mem nil (ssrbool.mem r1)

```

```

66         || ssrbool.in_mem nil (ssrbool.mem r2))%bool =
67         ssrbool.in_mem nil (ssrbool.mem (N.RS.Alt r1 r2)) =>
68     eq_ind_r
69     (fun _pattern_value_ : bool =>
70     (ssrbool.in_mem nil (ssrbool.mem r1) || _pattern_value_)%bool =
71     ssrbool.in_mem nil (ssrbool.mem (N.RS.Alt r1 r2)))
72     _evar_0_ IH2)
73     eq_refl))
74 (fun (r1 : N.RS.regex) (IH1 : N.has_eps r1 = ssrbool.in_mem nil (ssrbool.mem r1))
75 (r2 : N.RS.regex) (IH2 : N.has_eps r2 = ssrbool.in_mem nil (ssrbool.mem r2)) =>
76 (fun
77     _evar_0_ : (ssrbool.in_mem nil (ssrbool.mem r1) && N.has_eps r2)%bool =
78     ssrbool.in_mem nil (ssrbool.mem (N.RS.Seq r1 r2)) =>
79     eq_ind_r
80     (fun _pattern_value_ : bool =>
81     (_pattern_value_ && N.has_eps r2)%bool =
82     ssrbool.in_mem nil (ssrbool.mem (N.RS.Seq r1 r2)))
83     _evar_0_ IH1)
84 ((fun
85     _evar_0_ : (ssrbool.in_mem nil (ssrbool.mem r1) &&
86     ssrbool.in_mem nil (ssrbool.mem r2))%bool =
87     ssrbool.in_mem nil (ssrbool.mem (N.RS.Seq r1 r2)) =>
88     eq_ind_r
89     (fun _pattern_value_ : bool =>
90     (ssrbool.in_mem nil (ssrbool.mem r1) && _pattern_value_)%bool =
91     ssrbool.in_mem nil (ssrbool.mem (N.RS.Seq r1 r2)))
92     _evar_0_ IH2)
93 ((fun
94     _evar_0_ : (ssrbool.in_mem nil (ssrbool.mem r1) &&
95     ssrbool.in_mem nil (ssrbool.mem r2))%bool =
96     (N.RS.den r1 nil && N.RS.den r2 nil)%bool =>
97     eq_ind_r
98     (fun _pattern_value_ : bool =>
99     (ssrbool.in_mem nil (ssrbool.mem r1) && ssrbool.in_mem nil (ssrbool.mem
100 r2))%bool =
101     _pattern_value_)
102     _evar_0_ (Languages.conc_nil (A:=BoolSym.A) (N.RS.den r1) (N.RS.den r2)))
103     eq_refl)))
104 (fun (r : N.RS.regex) (_ : N.has_eps r = ssrbool.in_mem nil (ssrbool.mem r)) =>
105 ssreflect.ssr_have
106 (Languages.star_nil (A:=BoolSym.A) (N.RS.den r)
107 :
108 is_true (N.RS.star (N.RS.den r) nil))
109 (fun H : is_true (N.RS.star (N.RS.den r) nil) =>

```

```

109      (fun
110        (_evar_0_ : (fun b : bool =>
111          is_true b -> true = ssrbool.in_mem nil (ssrbool.mem (N.RS.Star
r))))
112          true)
113      (_evar_0_0 : (fun b : bool =>
114        is_true b -> true = ssrbool.in_mem nil (ssrbool.mem (N.RS.Star
r))))
115          false) =>
116      if N.RS.star (N.RS.den r) nil as b
117      return
118      ((fun b0 : bool =>
119        is_true b0 -> true = ssrbool.in_mem nil (ssrbool.mem (N.RS.Star r))) b)
120      then _evar_0_
121      else _evar_0_0) (fun H0 : is_true true => H0) (fun _ : is_true false =>
eq_refl) H))
122      (fun (r1 : N.RS.regex) (IH1 : N.has_eps r1 = ssrbool.in_mem nil (ssrbool.mem r1))
123        (r2 : N.RS.regex) (IH2 : N.has_eps r2 = ssrbool.in_mem nil (ssrbool.mem r2)) =>
124        (fun
125          _evar_0_ : (ssrbool.in_mem nil (ssrbool.mem r1) && N.has_eps r2)%bool =
126            ssrbool.in_mem nil (ssrbool.mem (N.RS.And r1 r2)) =>
127            eq_ind_r
128            (fun _pattern_value_ : bool =>
129              (_pattern_value_ && N.has_eps r2)%bool =
130              ssrbool.in_mem nil (ssrbool.mem (N.RS.And r1 r2)))
131            _evar_0_ IH1)
132          ((fun
133            _evar_0_ : (ssrbool.in_mem nil (ssrbool.mem r1) &&
134              ssrbool.in_mem nil (ssrbool.mem r2))%bool =
135              ssrbool.in_mem nil (ssrbool.mem (N.RS.And r1 r2)) =>
136              eq_ind_r
137              (fun _pattern_value_ : bool =>
138                (ssrbool.in_mem nil (ssrbool.mem r1) && _pattern_value_)%bool =
139                ssrbool.in_mem nil (ssrbool.mem (N.RS.And r1 r2)))
140              _evar_0_ IH2)
141            eq_refl))
142          (fun (r : N.RS.regex) (IH : N.has_eps r = ssrbool.in_mem nil (ssrbool.mem r)) =>
143            (fun
144              _evar_0_ : negb (ssrbool.in_mem nil (ssrbool.mem r)) =
145                ssrbool.in_mem nil (ssrbool.mem (N.RS.Neg r)) =>
146                eq_ind_r
147                (fun _pattern_value_ : bool =>
148                  negb _pattern_value_ = ssrbool.in_mem nil (ssrbool.mem (N.RS.Neg r)))
149              _evar_0_ IH)

```

```

150   eq_refl)
151   : forall r : N.RS.regex, N.has_eps r = ssrbool.in_mem nil (ssrbool.mem r)

```

Definition 3.6 (Semantic equivalence). *For regular expressions r, s , the relation $r \equiv s$ holds if $\mathcal{L}(r) = \mathcal{L}(s)$.*

In Rocq, equality of languages is extensional equality of the predicates, `lang_eq`, and two expressions are equivalent, `re_equiv`, when their denotations are equal; the development proves both to be equivalence relations.

Listing 6: Language equality and semantic equivalence (`RegexSemantics.v`).

```

1  (* Extensional equality of languages. *)
2  Definition lang_eq (L K : language) : Prop := L =i K.
3  Notation "L ≡ K" := (lang_eq L K) (at level 70).
4
5  (* 2 regexes are semantically equivalent when their denoted languages are equal. *)
6  Definition re_equiv (r s : regex) : Prop := r =i s.
7  Notation "r ≈ s" := (re_equiv r s) (at level 70).

```

Example. The expressions $a \cdot (b + c)$ and $a \cdot b + a \cdot c$ are semantically equivalent, $a \cdot (b + c) \equiv a \cdot b + a \cdot c$, because both denote $\{ab, ac\}$, and likewise $a + b \equiv b + a$ since both denote $\{a, b\}$.

Semantic equivalence is the relation that matters, but it is not finitely axiomatisable [12], [18] and is awkward to compute with directly. Following Brzozowski [3] and Coquand and Siles [6], the development instead uses a decidable, finitely generated approximation.

Definition 3.7 (Similarity and canonical form). *The similarity relation $\sim$ on regular expressions is generated by the following equational laws. For union it includes idempotence, commutativity and associativity, together with the unit law for $\emptyset$:*

$$\begin{aligned}
 r + r &\sim r, & r + s &\sim s + r, & (r + s) + t &\sim r + (s + t), \\
 r + \emptyset &\sim r, & \emptyset + r &\sim r.
 \end{aligned}$$

For concatenation it includes associativity, the annihilation laws for $\emptyset$, and the unit laws for ε :

$$\begin{aligned}
 (r \cdot s) \cdot t &\sim r \cdot (s \cdot t), \\
 \emptyset \cdot r &\sim \emptyset, & r \cdot \emptyset &\sim \emptyset, & \varepsilon \cdot r &\sim r, & r \cdot \varepsilon &\sim r.
 \end{aligned}$$

For the Kleene star it includes

$$\emptyset^* \sim \varepsilon, \quad \varepsilon^* \sim \varepsilon, \quad (r^*)^* \sim r^*.$$

For intersection it includes idempotence, commutativity and associativity, the annihilation laws for $\emptyset$, and the unit laws for $\top := \neg\emptyset$:

$$r \& r \sim r, \quad r \& s \sim s \& r, \quad (r \& s) \& t \sim r \& (s \& t),$$

$$r \& \emptyset \sim \emptyset, \quad \emptyset \& r \sim \emptyset, \quad r \& \top \sim r, \quad \top \& r \sim r.$$

Finally, complement is involutive, $\neg\neg r \sim r$.

Remark 3.1 (Two unrelated notions of “similarity”). *The word similarity is used in this thesis for two entirely different relations, and they should not be conflated. The relation $\sim$ of Definition 3.7 is the one meant wherever “similarity” appears without qualification in the crisp development of Sections 3.1–3.3. It is the Coquand–Siles similarity [6], a structural, two-valued relation between regular expressions, generated by the equational laws above and decided by comparing canonical forms. It changes nothing about symbol matching. It only identifies expressions that are equal up to associativity, commutativity, idempotence and the unit and annihilation laws. By contrast, the fuzzy similarity introduced later, in Section 3.4, is a graded $[0, 1]$ -valued relation between alphabet symbols, and, derived from it, between words, languages and expressions. It is the graded form of symbol equality that defines the fuzzy layer, and it is defined by a triangular norm rather than by equational laws. The two are independent. The crisp similarity $\sim$ is in force throughout the fuzzy development as well, where it continues to canonicalise expressions, while the fuzzy similarity acts on symbols. Where confusion is possible the qualifiers “Coquand–Siles” (or “structural”) and “fuzzy” (or “graded”) are used.*

In Rocq the similarity relation is decided by normalising both expressions with a function `canonize` and comparing the results structurally. The normaliser is built from *smart constructors* that build the laws above into the way expressions are constructed. For instance `mkStar` realises $\emptyset^* \sim \varepsilon$, $\varepsilon^* \sim \varepsilon$ and $(r^*)^* \sim r^*$, while `mkPlus` (not shown) sorts and deduplicates its arguments to enforce associativity, commutativity and idempotence of $+$. Similarity is then similar $r \sim s$, the structural equality of the two normal forms.

Listing 7: Smart constructors and normalisation (`Canonicalization.v`).

```

1 Definition mkStar (r : regex) : regex :=
2   match r with
3   | Empty => Eps
4   | Eps   => Eps

```

```

5   | Star r' => Star r'
6   | _ => Star r
7   end.
8
9   Fixpoint canonize (r : regex) : regex :=
10  match r with
11  | Empty => Empty
12  | Eps   => Eps
13  | Char a => Char a
14  | Alt r1 r2 => mkPlus (canonize r1) (canonize r2)
15  | Seq r1 r2 => mkConc (canonize r1) (canonize r2)
16  | Star r1   => mkStar (canonize r1)
17  | And r1 r2 => mkAnd  (canonize r1) (canonize r2)
18  | Neg r1    => mkNot  (canonize r1)
19  end.
20
21 Definition nf := canonize.
22 Definition similar (r s : regex) : bool :=
23   eq_regex (nf r) (nf s).

```

Lemma 3.1 (Canonicalisation/Normalisation). *The equations of Definition 3.7 determine a canonicalisation function nf on regular expressions such that*

$$r \sim s \iff \text{nf}(r) = \text{nf}(s),$$

where $=$ is syntactic (structural) equality. The canonical form $\text{nf}(r)$ is written $[r]$.

Proof. This is the formalised theorem `similar_iff_nf_eq` of `CanonicalizationCorrectness.v`, in which nf is `canonize` and $\sim$ is the boolean `similar` of Listing 7. $\square$

Listing 8: Canonicalisation/normalisation, formalised (`CanonicalizationCorrectness.v`).

```

1 Theorem similar_iff_nf_eq (r s : regex) :
2   similar r s <-> nf r = nf s.
3 Proof.
4   split.
5   - move=> H.
6     exact: eq_regex_sound H.
7   - move=> H.
8     exact: eq_regex_complete H.
9 Qed.

```

Lemma 3.2 (Soundness of normalisation). *If $r \sim s$ then $\mathcal{L}(r) = \mathcal{L}(s)$, ergo $r \equiv s$.*

Proof. This is the formalised theorem `similar_sound` of `CanonicalizationCorrectness.v`, where `re_equiv` is the semantic equivalence of Listing 6. It rests on `canonize_correct`, which states that normalisation preserves the denoted language, so that $\mathcal{L}(\text{nf}(r)) = \mathcal{L}(r)$ for every r . $\square$

Listing 9: Soundness of normalisation, formalised (`CanonicalizationCorrectness.v`).

```

1 Theorem canonize_correct (r : regex) (w : word) :
2   (w \in canonize r) = (w \in r).
3 Proof.
4   elim: r w => [| |a |r1 IH1 r2 IH2 |r1 IH1 r2 IH2 |r IH |r1 IH1 r2 IH2 |r IH] w //=.
5   - (* Alt *)
6     rewrite (mkPlus_correct (canonize r1) (canonize r2) w).
7     by rewrite IH1 IH2.
8   - (* Seq *)
9     rewrite (mkConc_correct (canonize r1) (canonize r2) w).
10    exact: (Languages.conc_ext (A:=A) IH1 IH2 w).
11  - (* Star *)
12    rewrite (mkStar_correct (canonize r) w).
13    exact: (Languages.star_ext (A:=A) IH w).
14  - (* And *)
15    rewrite (mkAnd_correct (canonize r1) (canonize r2) w).
16    by rewrite IH1 IH2.
17  - (* Neg *)
18    rewrite (mkNot_correct (canonize r) w).
19    exact: ((RS.re_equiv_Neg IH) w).
20 Qed.
21
22 Theorem similar_sound (r s : regex) :
23   similar r s -> r ≈ s.
24 Proof.
25   move=> H w.
26   have Hcanon : canonize r = canonize s := eq_regex_sound H.
27   rewrite -(canonize_correct r w).
28   rewrite -(canonize_correct s w).
29   by rewrite Hcanon.
30 Qed.

```

Example. Idempotence, commutativity and associativity of $+$ give $a + (b + a) \sim a + b$, so the two share a canonical form, $[a + (b + a)] = [a + b]$. Similarly the star laws give $(a^*)^* \sim a^*$ and the unit law gives $\varepsilon \cdot a \sim a$, so $[(a^*)^*] = [a^*]$ and $[\varepsilon \cdot a] = [a]$.

The significance of similarity, due to Brzozowski, is that although no *finite* equational system captures semantic equivalence (the star operator can only be approximated finitely), this finite approximation is already enough to make the set of derivative states finite. For example a^* and $(\varepsilon + a)^*$ are semantically equivalent but not similar. The construction of `nf` and the two lemmas above build on the development of Coquand and Siles [6].

3.2 Derivatives of Regular Languages and Regular Expressions

The derivative of a language with respect to a symbol a collects the suffixes of the words that begin with a .

Definition 3.8 (Semantic derivative). *The derivative of a language $L \subseteq \Sigma^*$ with respect to a symbol $a \in \Sigma$ is*

$$\partial_a(L) = \{w \in \Sigma^* \mid aw \in L\}.$$

For a word $u = aw$ with $a \in \Sigma$ and $w \in \Sigma^$, the derivative is extended by $\partial_u(L) = \partial_w(\partial_a(L))$ and $\partial_\varepsilon(L) = L$.*

Example. Let $\Sigma = \{a, b\}$ and $L = \{aab, bb, baa, a\}$. Then $\partial_a(L) = \{ab, \varepsilon\}$. From aab one obtains ab , from a one obtains ε , and bb, baa are discarded because they do not start with a .

On the language side the formalisation defines the derivative by a single symbol, `dlang_char`, as the predicate that holds of w exactly when $a :: w$ is in L , and the derivative by a word, `dlang`, as the predicate that holds of w when $u ++ w$ is in L .

Listing 10: Semantic (language) derivatives (`Derivatives.v`).

```

1 (* semantic derivatives *)
2 Definition dlang_char (a : A) (L : language) : language :=
3   fun w => L (a :: w).
4
5 Definition dlang (u : word) (L : language) : language :=
6   fun w => L (u ++ w).
```

The same operation can be computed directly on regular expressions, without first interpreting them as languages. This is the syntactic derivative.

Definition 3.9 (Syntactic derivative w.r.t. a symbol). *For a regular expression r over Σ and $a \in \Sigma$, the*

derivative $\mathcal{D}_a(r)$ is defined recursively by

$$\begin{aligned}
\mathcal{D}_a(\emptyset) &= \emptyset, & \mathcal{D}_a(\varepsilon) &= \emptyset, \\
\mathcal{D}_a(a) &= \varepsilon, & \mathcal{D}_a(b) &= \emptyset \quad (a \neq b), \\
\mathcal{D}_a(r + s) &= \mathcal{D}_a(r) + \mathcal{D}_a(s), & \mathcal{D}_a(r^*) &= \mathcal{D}_a(r) \cdot r^*, \\
\mathcal{D}_a(r \& s) &= \mathcal{D}_a(r) \& \mathcal{D}_a(s), & \mathcal{D}_a(\neg r) &= \neg(\mathcal{D}_a(r)), \\
\mathcal{D}_a(r \cdot s) &= \mathcal{D}_a(r) \cdot s + \nu(r) \cdot \mathcal{D}_a(s).
\end{aligned}$$

The derivative with respect to a word is $\mathcal{D}_\varepsilon(r) = r$ and $\mathcal{D}_{aw}(r) = \mathcal{D}_w(\mathcal{D}_a(r))$.

The symbol derivative $\mathcal{D}_a(\cdot)$ is the function `D_char`. The `Seq` clause reproduces the product rule $\mathcal{D}_a(r \cdot s) = \mathcal{D}_a(r) \cdot s + \nu(r) \cdot \mathcal{D}_a(s)$, with `nu r1` guarding the second summand. The word derivative `D` folds `D_char` along the word.

Listing 11: Syntactic derivatives (`Derivatives.v`).

```

1 (* syntactic derivatives *)
2 Fixpoint D_char (a : A) (r : regex) : regex :=
3   match r with
4   | Empty      => Empty
5   | Eps        => Empty
6   | Char b     => if a == b then Eps else Empty
7   | Alt r1 r2  => Alt (D_char a r1) (D_char a r2)
8   | And r1 r2  => And (D_char a r1) (D_char a r2)
9   | Seq r1 r2  => Alt (Seq (D_char a r1) r2)
10                  (Seq (nu r1) (D_char a r2))
11   | Star r1    => Seq (D_char a r1) (Star r1)
12   | Neg r1     => Neg (D_char a r1)
13   end.
14
15 Fixpoint D (u : word) (r : regex) : regex :=
16   if u is a :: v then D v (D_char a r) else r.

```

Example. $\mathcal{D}_a(a + b) = \mathcal{D}_a(a) + \mathcal{D}_a(b) = \varepsilon + \emptyset$, which canonicalises to ε . For the star, $\mathcal{D}_a(a^*) = \mathcal{D}_a(a) \cdot a^* = \varepsilon \cdot a^*$, and this is similar to a^* rather than syntactically equal to it, $\varepsilon \cdot a^* \sim a^*$. Continuing in the same way, every derivative of a^* by a power of a is again a^* up to similarity, which is why canonicalisation collapses them to a single state in the automaton construction below.

The first main result establishes that the two derivatives, the semantic and the syntactic one, describe the same operation.

Theorem 3.1 (Semantic–syntactic correspondence). *For every regular expression r over Σ and every word $u \in \Sigma^*$,*

$$\mathcal{L}(\mathcal{D}_u(r)) = \partial_u(\mathcal{L}(r)).$$

Proof. The correspondence is established in Rocq, as the lemma `D_correct` shown in Listing 12. It states that a word w is in the derivative $\mathcal{D}_u r$ exactly when $u ++ w$ is in r , which is the membership form of the equality $\mathcal{L}(\mathcal{D}_u(r)) = \partial_u(\mathcal{L}(r))$. The single-symbol case is `D_char_correct`, proved by structural induction on the expression with `Seq` and `Star` the main cases. From it `D_correct` follows by a short induction on the word. $\square$

Listing 12: The correspondence theorem (`Derivatives.v`).

```

1 Lemma D_char_correct : forall a r w, (w \in D_char a r) = ((a :: w) \in r).
2 Proof.
3   move=> a; elim=> //.
4
5   - (* Char *)
6     move=> b w.
7     rewrite /in_mem /=.
8     case: ifP=> [/eqP ->|hab] /=.
9     + by rewrite /eps /atom /Languages.eps /Languages.atom
10        /= eqseq_cons eqxx /=.
11     + move/negPf: hab=>hab.
12       by rewrite /void /atom /Languages.void /Languages.atom
13          /= eqseq_cons hab /=.
14
15   - (* Alt *)
16     move=> r1 IH1 r2 IH2 w.
17     rewrite /in_mem /= /plus /Languages.plus /=.
18     exact: (congr2 orb (IH1 w) (IH2 w)).
19
20   (* Seq and Star are the non-trivial cases. Seq splits a :: w as a
21      concatenation, the head symbol being consumed by D_a(r1) or, when
22      r1 is nullable, by D_a(r2); Star peels the first nonempty block
23      with star_cons_split and applies the induction hypothesis. The full
24      concP bookkeeping is in Derivatives.v [10]. *)
25
26   - (* And *)
27     move=> r1 IH1 r2 IH2 w.
28     rewrite /in_mem /= /prod /Languages.prod /=.
29     exact: (congr2 andb (IH1 w) (IH2 w)).
30
31   - (* Neg *)

```

```

32   move=> r IHr w.
33   rewrite /in_mem /= /compl /Languages.compl /=.
34   exact: (congr1 negb (IHr w)).
35 Qed.
36
37 Lemma D_correct : forall u r w, (w \in D u r) = ((u ++ w) \in r).
38 Proof.
39   elim=> [|a u IHu] r w //=.
40   rewrite IHu D_char_correct.
41   by rewrite -cat_cons.
42 Qed.

```

A central consequence of the correspondence is that derivatives keep a regular language regular.

Theorem 3.2 (Regularity of derivatives). *If $L \subseteq \Sigma^*$ is regular, then $\partial_u(L)$ is regular for every word $u \in \Sigma^*$.*

Proof. In Rocq this is the theorem `derivative_regular` of Listing 13. A language is regular when it is denoted by some expression, that is, when exists r , $L = i \text{ den } r$. If r denotes L , then Theorem 3.1, in the form `D_correct`, makes $\mathcal{D}_u(r)$ an expression that denotes $\partial_u(L)$, because $\partial_u(L) = \mathcal{L}(\mathcal{D}_u(r))$. Ergo $\partial_u(L)$ is again denoted by an expression and is regular. $\square$

Listing 13: Regularity of derivatives, formalised (`Derivatives.v`).

```

1 Definition regular (L : language) : Prop := exists r : regex, L =i den r.
2
3 Theorem derivative_regular (L : language) (u : word) :
4   regular L -> regular (dlang u L).
5 Proof.
6   move=> [r Hr].
7   exists (D u r) => w.
8   rewrite /dlang.
9   rewrite /dlang /in_mem /=.
10  have HL : L (u ++ w) = ((u ++ w) \in r).
11    by move: (Hr (u ++ w)); rewrite /in_mem /=.
12  rewrite HL.
13  by rewrite -(D_correct u r w).
14 Qed.

```

Theorem 3.3 (Finiteness of derivatives, Brzozowski [3, Theorem 5.2]). *Let Σ be finite and $L \subseteq \Sigma^*$ regular. Then the set $\{\partial_u(L) \mid u \in \Sigma^*\}$ is finite.*

Proof. By Theorem 3.1 it suffices to bound the set $\{\mathcal{L}(\mathcal{D}_u(r)) \mid u \in \Sigma^*\}$, which is done by induction on the number N of regular operators in r .

Base cases ($N = 0$). If $r = \emptyset$ then $\mathcal{D}_u(\emptyset) = \emptyset$ for all u , so the collection is $\{\emptyset\}$. If $r = \varepsilon$ then $\mathcal{D}_\varepsilon(\varepsilon) = \varepsilon$ and $\mathcal{D}_a(\varepsilon) = \emptyset$, giving $\{\{\varepsilon\}, \emptyset\}$. If $r = a \in \Sigma$ then $\mathcal{D}_\varepsilon(a) = a$, $\mathcal{D}_a(a) = \varepsilon$ and $\mathcal{D}_b(a) = \emptyset$ for $b \neq a$, a finite collection.

Inductive step ($N > 0$). Assume the claim for all expressions with at most N operators, and let r have $N + 1$ operators with immediate subexpressions p, q (each with at most N).

(1) *Boolean combinations.* If $r = p + q$ then $\mathcal{L}(\mathcal{D}_u(r)) = \mathcal{L}(\mathcal{D}_u(p)) \cup \mathcal{L}(\mathcal{D}_u(q))$, and by the induction hypothesis each term ranges over a finite set, so only finitely many unions occur. The cases $r = p \& q$ and $r = \neg p$ are identical with $\cap$ and complement in place of $\cup$.

(2) *Concatenation.* For $r = p \cdot q$ a straightforward induction on the length of $u = a_1 \cdots a_n$ from the product rule yields

$$\mathcal{D}_u(p \cdot q) = \mathcal{D}_u(p) \cdot q + \sum_{i=0}^{n-1} \nu(\mathcal{D}_{a_1 \cdots a_i}(p)) \cdot \mathcal{D}_{a_{i+1} \cdots a_n}(q),$$

where the sum denotes a finite union via $+$. Each summand is built from a derivative of p or of q (the factors $\nu(\cdot)$ contribute only ε or $\emptyset$), so by the induction hypothesis there are finitely many possible languages for each term and so finitely many possible unions.

(3) *Kleene star.* For $r = p^*$ induction on $n \geq 1$ shows that $\mathcal{D}_u(p^*)$ is a finite union of terms $(\mathcal{D}_t(p)) \cdot p^*$. The base case is $\mathcal{D}_a(p^*) = \mathcal{D}_a(p) \cdot p^*$. For the step, if $\mathcal{D}_u(p^*) = \sum_j (\mathcal{D}_{t_j}(p)) \cdot p^*$ then applying the product rule to each summand gives

$$\mathcal{D}_a((\mathcal{D}_{t_j}(p)) \cdot p^*) = (\mathcal{D}_a(\mathcal{D}_{t_j}(p))) \cdot p^* + \nu(\mathcal{D}_{t_j}(p)) \cdot (\mathcal{D}_a(p)) \cdot p^*,$$

again a union of terms of the same form. By the induction hypothesis on p the factors $\mathcal{D}_t(p)$ range over finitely many languages, so only finitely many such unions arise.

In every case the collection $\{\mathcal{L}(\mathcal{D}_u(r)) \mid u \in \Sigma^*\}$ is finite, and so is $\{\partial_u(L) \mid u \in \Sigma^*\}$. $\square$

3.3 Finite Automata

The finiteness result of the previous subsection lets us turn a regular expression into a finite automaton. The canonical derivatives are the states and differentiation is the transition function. Each state stands for the language left after reading some prefix of the input. We compute the transitions on canonical representatives, so the number of states stays finite.

Definition 3.10 (Derivative DFA). *Let $\text{nf}(\cdot)$ be the normalisation of Lemma 3.1, and write $[r] := \text{nf}(r)$.*

The derivative DFA of a regular expression r is $M = \langle Q, \Sigma, \delta, q_0, F \rangle$ with

$$Q = \{ [\mathcal{D}_u(r)] \mid u \in \Sigma^* \}, \quad q_0 = [r], \quad F = \{ q \in Q \mid \nu(q) = \varepsilon \},$$

$$\delta(q, a) = [\mathcal{D}_a(q)],$$

where Σ is the alphabet of r .

In Rocq the automaton is a dependent record whose state type Q is left abstract, with an initial state q_0 , a transition function δ and a boolean acceptance test F . The derivative DFA instantiates this record by taking the states to be regular expressions. The initial state is `canonize r`, the transition canonicalises the one-step derivative `canonize (D_char a q)`, and a state is accepting exactly when it is nullable, that is, when `has_eps` holds of it.

Listing 14: The DFA record and the constructed derivative DFA (`Automata.v`).

```

1 Record dfa := {
2   Q      : Type;
3   q0     : Q;
4   delta  : Q -> A -> Q;
5   F      : Q -> bool
6 }.
7
8 Definition constructed_dfa (r : regex) : dfa :=
9   { | Q      := regex;
10     q0      := canonize r;
11     delta   := fun q a => canonize (D_char a q);
12     F       := has_eps | }.

```

States are compared by structural equality of their normal forms, the boolean `regex_eqb`, not by their languages. The construction keeps taking derivatives and adds a new state whenever it meets a canonical form it has not seen before. In Rocq, in `AutomataConstruction.v`, this is the recursive function `goto`. It computes the canonical derivative `q_a := canonical_derivative a q`, that is $[\mathcal{D}_a(q)]$. If `q_a` is already among the discovered states it records the transition. Otherwise it adds `q_a` and recurses on every symbol of the alphabet. The extra parameter `fuel` is discussed in Remark 3.3 below.

Listing 15: The recursive exploration `goto` (`AutomataConstruction.v`).

```

1 Fixpoint goto (fuel : nat) (q : state) (a : A)
2   (Q : seq state) (delta : seq transition)
3   {struct fuel} : seq state * seq transition :=
4   match fuel with

```

```

5 | 0 => (Q, delta)
6 | fuel'.+1 =>
7   let q_a := canonical_derivative a q in
8   if mem_state q_a Q then
9     (Q, add_transition q a q_a delta)
10  else
11    let Q1 := add_state q_a Q in
12    let delta1 := add_transition q a q_a delta in
13    let fix explore (Sigma : seq A) (Qacc : seq state)
14      (deltaacc : seq transition) {struct Sigma}
15      : seq state * seq transition :=
16      if Sigma is a' :: Sigma' then
17        let '(Qnext, deltanext) := goto fuel' q_a a' Qacc deltaacc in
18        explore Sigma' Qnext deltanext
19      else (Qacc, deltaacc)
20    in explore alphabet Q1 delta1
21 end.

```

The driver `construct_dfa` mirrors the abstract `dfa( $r$ )` procedure. It starts from $q_0 = [r] = \text{canonize } r$, runs `goto` from q_0 on every symbol, and marks the nullable states as accepting (`final_states` keeps the ones where `has_eps` holds). Where the abstract record of Listing 14 leaves the state type Q abstract, here the reachable states become a concrete list $Q : \text{seq state}$ and the transitions a lookup table $\text{delta} : \text{seq transition}$. This is the finite realisation of that abstract automaton.

Listing 16: The driver `construct_dfa` (`AutomataConstruction.v`).

```

1 Definition construct_dfa (fuel : nat) (r : regex) : dfa :=
2   let q0 := canonize r in
3   let fix build_from_initial (Sigma : seq A) (Q : seq state)
4     (delta : seq transition) {struct Sigma}
5     : seq state * seq transition :=
6     if Sigma is a :: Sigma' then
7       let '(Qnext, deltanext) := goto fuel q0 a Q delta in
8       build_from_initial Sigma' Qnext deltanext
9     else (Q, delta)
10  in
11  let '(Q, delta) := build_from_initial alphabet [:: q0] [::] in
12  { | Q := Q;
13    Sigma := alphabet;
14    delta := delta;
15    q0 := q0;
16    F := final_states Q | }.

```

Remark 3.2 (Termination). *The exploration adds a state only when a new canonical form appears, and it halts once none does. Termination is the claim that this halt is reached. Since Σ is finite, each state branches finitely, so termination reduces to the set of canonical forms being finite. In a constructive setting “finite” is not a single notion, and which notion holds decides whether a halting procedure exists. Coquand and Siles [6] settle this.*

Brzowski’s result that any regular expression has only a finite number of derivatives is purely existential. [...] In type theory, this will correspond to the fact that we can prove that the set of all derivatives of E is inductively finite up to similarity, but we can not prove without a further hypothesis on similarity (namely that similarity is closed under derivatives) that this set is Kuratowski finite up to similarity. On the other hand, we can always show that the set of derivatives is Kuratowski finite up to equivalence.

Remark 3.3 (The fuel argument). *The functions above carry an extra counter, `fuel`, that the abstract procedure does not need. Rocq requires it. Rocq models only total functions, so every term terminates, and ergo Rocq is not a full-fledged programming language. Rocq accepts a recursive function only when it can see that the function terminates, and it sees this by checking that each call works on a smaller piece of some input. The exploration does not fit this pattern. `goto` calls itself on the new derivative `q_a`, which is not a smaller piece of anything it was given. We know it terminates, because there are only finitely many states, but Rocq cannot see that on its own. The counter makes it visible. `goto` starts with some `fuel` and spends one unit each time it reaches a new state, and it terminates when the fuel runs out. The fuel only goes down, so Rocq can tell the function terminates.*

The fuel bounds how far the exploration goes. The reachable states are finite, and in the worst case their number grows exponentially with the size of r . So giving `goto` fuel equal to that exponential bound is enough to reach every state.

Theorem 3.4 (DFA correctness). *Let r be a regular expression over Σ , and let $M = \langle Q, \Sigma, \delta, q_0, F \rangle$ be the DFA constructed from r . Then M and r define the same language:*

$$\mathcal{L}(M) = \mathcal{L}(r).$$

Equivalently, for every word $u \in \Sigma^$,*

$$M \text{ accepts } u \iff u \in \mathcal{L}(r).$$

This is proved in Rocq as the theorem `dfa_correctness` of Listing 17, where `=i` (the language equality written `≡` in the source) is extensional equality of language predicates. The run of the automaton on a word is the extended transition `delta_star`. The proof reduces acceptance to membership in three rewrites. `nullable_correct` turns the accepting test `has_eps` into membership of the empty

word, the lemma `delta_star_constructed_lang` replaces the state reached after reading u by the corresponding derivative, and `canonize_correct_local` discharges the remaining goal because normalisation preserves the denoted language.

Listing 17: The run of the automaton and the formalised correctness proof (`Automata.v`).

```

1 Fixpoint delta_star (M : dfa) (q : Q M) (w : word) : Q M :=
2   if w is a :: w' then @delta_star M (@delta M q a) w' else q.
3
4 Theorem dfa_correctness (r : regex) :
5   dfa_lang (constructed_dfa r) ≡ den r.
6 Proof.
7   move=> u.
8   change (has_eps (delta_star (constructed_dfa r) (q0 (constructed_dfa r)) u) =
9     (u \in r)).
10  rewrite nullable_correct.
11  rewrite (delta_star_constructed_lang r (canonize r) u [::]).
12  rewrite cats0.
13  exact: canonize_correct_local.
14 Qed.

```

Example. Let $\Sigma = \{a, b\}$ and run the construction on the restricted regular expression $r = (a + b)^* \cdot a$. The construction of Listings 15 and 16 starts from the initial state $q_0 = [r]$ and, for each state it discovers, takes the canonical derivative with respect to every symbol. In the Rocq formulation each such step is `canonize (D_char a q)`, and a state is accepting when `has_eps` holds of it.

Begin with $q_0 = [(a + b)^* \cdot a]$. Differentiating by a , applying the product rule and then the star rule,

$$\begin{aligned}
 \mathcal{D}_a((a + b)^* \cdot a) &= \mathcal{D}_a((a + b)^*) \cdot a + \nu((a + b)^*) \cdot \mathcal{D}_a(a) \\
 &= (\mathcal{D}_a(a + b) \cdot (a + b)^*) \cdot a + \varepsilon \cdot \mathcal{D}_a(a) \\
 &= ((\varepsilon + \emptyset) \cdot (a + b)^*) \cdot a + \varepsilon \cdot \varepsilon,
 \end{aligned}$$

where the last line uses $\mathcal{D}_a(a + b) = \mathcal{D}_a(a) + \mathcal{D}_a(b) = \varepsilon + \emptyset$ and $\mathcal{D}_a(a) = \varepsilon$. This raw expression is not normalised; canonicalisation rewrites it step by step with the laws of Definition 3.7,

$$\begin{aligned}
 \varepsilon + \emptyset &\sim \varepsilon, \\
 \varepsilon \cdot (a + b)^* &\sim (a + b)^*, \\
 \varepsilon \cdot \varepsilon &\sim \varepsilon,
 \end{aligned}$$

so that the raw derivative is similar, but not syntactically equal, to a smaller expression,

$$\mathcal{D}_a(r) \sim (a + b)^* \cdot a + \varepsilon,$$

and the transition records its canonical form, $\delta(q_0, a) = [\mathcal{D}_a(r)] = (a + b)^* \cdot a + \varepsilon$, a canonical form not seen before, recorded as a new state q_1 .

Differentiating q_0 by b instead, the same rules give

$$\mathcal{D}_b((a + b)^* \cdot a) = ((\emptyset + \varepsilon) \cdot (a + b)^*) \cdot a + \varepsilon \cdot \emptyset.$$

Canonicalising as before gives $\mathcal{D}_b(r) \sim r$, so $\delta(q_0, b) = [\mathcal{D}_b(r)] = [r] = q_0$, an already-known state.

The only state still to process is $q_1 = [(a + b)^* \cdot a + \varepsilon]$. By the union rule and $\mathcal{D}_x(\varepsilon) = \emptyset$, the exact derivatives are

$$\mathcal{D}_a(q_1) = \mathcal{D}_a((a + b)^* \cdot a) + \mathcal{D}_a(\varepsilon) = \mathcal{D}_a((a + b)^* \cdot a) + \emptyset,$$

$$\mathcal{D}_b(q_1) = \mathcal{D}_b((a + b)^* \cdot a) + \mathcal{D}_b(\varepsilon) = \mathcal{D}_b((a + b)^* \cdot a) + \emptyset.$$

The computations above already showed $\mathcal{D}_a((a + b)^* \cdot a) \sim q_1$ and $\mathcal{D}_b((a + b)^* \cdot a) \sim q_0$, and $s + \emptyset \sim s$, so $\mathcal{D}_a(q_1) \sim q_1$ and $\mathcal{D}_b(q_1) \sim q_0$. Taking canonical forms, $\delta(q_1, a) = [\mathcal{D}_a(q_1)] = q_1$ and $\delta(q_1, b) = [\mathcal{D}_b(q_1)] = q_0$. No new canonical form appears, the worklist is empty, and the construction stops.

It remains to mark the accepting states, those that are nullable. Here $\nu(q_0) = \nu((a + b)^*) \& \nu(a) = \varepsilon \& \emptyset = \emptyset$, so q_0 is not accepting, while $\nu(q_1) = \nu((a + b)^* \cdot a) + \nu(\varepsilon) = \emptyset + \varepsilon = \varepsilon$, so q_1 is accepting. The construction ergo returns the two-state automaton with the transition table

	a	b	accepting
q_0	q_1	q_0	no
q_1	q_1	q_0	yes

drawn in Figure 1. By Theorem 3.4 it recognises exactly $\mathcal{L}(r)$, the words over $\{a, b\}$ that end in a .

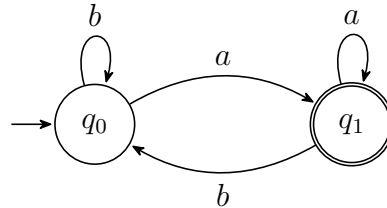
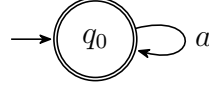


Figure 1: The derivative automaton of $r = (a + b)^* \cdot a$, with initial state $q_0 = [(a + b)^* \cdot a]$ and accepting state $q_1 = [(a + b)^* \cdot a + \varepsilon]$. Each edge is a canonical one-step derivative, $\delta(q, x) = [\mathcal{D}_x(q)]$.

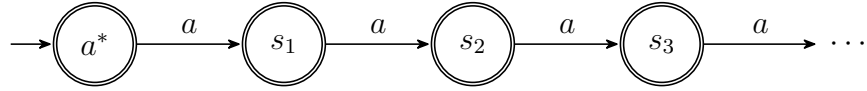
Example (with and without canonicalisation). This example shows exactly why canonicalisation

matters. Take $\Sigma = \{a\}$ and $r = a^*$.

With canonicalisation, the start state is $q_0 = [a^*] = a^*$, and $\mathcal{D}_a(a^*) = \varepsilon \cdot a^* \sim a^*$, so $\delta(q_0, a) = [a^*] = q_0$. The queue empties at once and the construction returns the one-state automaton.



Without canonicalisation, states are compared as raw syntax. Reading a gives $s_1 = \varepsilon \cdot a^*$, then $s_2 = \mathcal{D}_a(s_1)$, then s_3 , each larger than the last. Every s_i denotes a^* and every pair is similar, yet no two are syntactically equal, so the construction never terminates. Canonicalisation collapses this chain to the single representative a^* .



3.4 Fuzzy Proximity and Similarity Relations

The fuzzy setting compares symbols by degree rather than by equality. The first ingredient is a way of combining degrees of similarity, namely a T-norm [16].

Definition 3.11 (T-norm). *A T-norm is a binary operation $\otimes : [0, 1] \times [0, 1] \rightarrow [0, 1]$ such that, for all $x, y, z \in [0, 1]$:*

1. **Commutativity:** $x \otimes y = y \otimes x$;
2. **Associativity:** $(x \otimes y) \otimes z = x \otimes (y \otimes z)$;
3. **Monotonicity:** if $x \leq y$, then $x \otimes z \leq y \otimes z$;
4. **Neutral element:** $x \otimes 1 = x$.

We work with Gödel's T-norm.

The development formalises a T-norm as a record `T_norm` bundling the axioms above, together with the requirement that the operation maps $[0, 1]$ back into $[0, 1]$, and provides Gödel's minimum `smn` as the instance `godel_t_norm`, whose record fields are proved by the corresponding lemmas about `smn`.

Listing 18: T-norms and Gödel's minimum (`SimilarityRelations.v`).

```

1 Definition simval := R.
2
3 Definition in01 (x : simval) : Prop := 0 <= x <= 1.
4

```

```

5 Record T_norm : Type := {
6   tnorm : simval -> simval -> simval;
7   tnorm_closed :
8     forall x y, in01 x -> in01 y -> in01 (tnorm x y);
9   tnorm_comm :
10    forall x y, in01 x -> in01 y -> tnorm x y = tnorm y x;
11   tnorm_assoc :
12    forall x y z, in01 x -> in01 y -> in01 z ->
13      tnorm (tnorm x y) z = tnorm x (tnorm y z);
14   tnorm_mono_l :
15    forall x y z, in01 x -> in01 y -> in01 z ->
16      x <= y -> tnorm x z <= tnorm y z;
17   tnorm_neutral_r :
18    forall x, in01 x -> tnorm x 1 = x
19 }.
20
21 (* Godel's minimum T-norm *)
22 Definition smin (x y : simval) : simval :=
23   if Rle_dec x y then x else y.
24
25 Definition godel_t_norm : T_norm :=
26   { | tnorm := smin;
27     tnorm_closed := smin_in010;
28     tnorm_comm := fun x y _ => smin_comm0 x y;
29     tnorm_assoc := fun x y z _ _ => smin_assoc0 x y z;
30     tnorm_mono_l := fun x y z _ _ Hxy => @smin_mono_l0 x y z Hxy;
31     tnorm_neutral_r := smin1r |}.

```

Definition 3.12 (Fuzzy relation, cut, proximity, similarity).

1. A binary fuzzy relation on a set S is a mapping $\mathcal{R} : S \times S \rightarrow [0, 1]$. If $\mathcal{R}$ is a fuzzy relation on S and μ is a number $0 < \mu \leq 1$ (a cut value), then the μ -cut of $\mathcal{R}$, denoted $\mathcal{R}_\mu$, is the crisp relation

$$\mathcal{R}_\mu := \{(s_1, s_2) \mid \mathcal{R}(s_1, s_2) \geq \mu\}.$$

2. A fuzzy relation $\mathcal{R}$ on S is a proximity relation if it is reflexive and symmetric:

Reflexivity: $\mathcal{R}(s, s) = 1$ for all $s \in S$;

Symmetry: $\mathcal{R}(s_1, s_2) = \mathcal{R}(s_2, s_1)$ for all $s_1, s_2 \in S$.

3. Let $\otimes$ be a left-continuous T-norm on $[0, 1]$. A proximity relation is a (T-)similarity relation iff it satisfies the reverse triangle inequality with respect to $\otimes$:

Reverse triangle inequality: $\mathcal{R}(s_1, s_2) \geq \mathcal{R}(s_1, s) \otimes \mathcal{R}(s, s_2)$ for all $s_1, s_2, s \in S$.

With Gödel’s minimum T-norm as $\otimes$, the symbol $\min$ is often written instead of $\otimes$. These notions are those used in similarity-based approximate reasoning [19]. In Rocq the cut is `mu_cut`, a proximity is the record `is_proximity`, and a similarity relation with respect to a given T-norm `T` is the record `is_T_similarity T`, which records that `T` is left-continuous and that the reverse triangle inequality holds with respect to `tnorm T`.

Example. Take $\Sigma = \{a, b, c, d\}$ and let $\mathcal{R}$ assign $\mathcal{R}(x, x) = 1$ for every x , $\mathcal{R}(a, c) = \mathcal{R}(c, a) = 0.9$, $\mathcal{R}(b, d) = \mathcal{R}(d, b) = 0.75$, and 0 to every other distinct pair. This $\mathcal{R}$ is a proximity relation, since it is reflexive and symmetric, and a similarity relation under the minimum, since the only nontrivial chains pass through a repeated symbol. Its cut depends on the threshold. At $\mu = 0.8$ the cut $\mathcal{R}_{0.8}$ relates a with c but leaves b and d apart, whereas at $\mu = 0.7$ the cut $\mathcal{R}_{0.7}$ relates both a with c and b with d .

Listing 19: Fuzzy relations, cut, proximity, and T -similarity (`SimilarityRelations.v`).

```

1 Definition cut_value (mu : simval) : Prop := 0 < mu /\ mu <= 1.
2
3 Section FuzzyRelationLayer.
4   Variable S : Type.
5
6   (* Fuzzy relation *)
7   Definition fuzzy_rel := S -> S -> simval.
8
9   (* Fuzzy relation range *)
10  Definition is_fuzzy_rel (R : fuzzy_rel) : Prop :=
11    forall s1 s2, in01 (R s1 s2).
12
13  (* mu-cut *)
14  Definition mu_cut (mu : simval) (R : fuzzy_rel) : S -> S -> Prop :=
15    fun s1 s2 => mu <= R s1 s2.
16
17  (* Proximity relation *)
18  Record is_proximity (R : fuzzy_rel) : Prop := {
19    prox_range : is_fuzzy_rel R;
20    prox_refl : forall s, R s s = 1;
21    prox_sym : forall s1 s2, R s1 s2 = R s2 s1
22  }.
23
24  (* T-similarity relation *)
25  Record is_T_similarity (T : T_norm) (R : fuzzy_rel) : Prop := {
26    tsim_left_continuous : left_continuous_on_01 (tnorm T);
27    tsim_prox : is_proximity R;
28    tsim_trans : forall s1 s2 s,

```

```

29   tnorm T (R s1 s) (R s s2) <= R s1 s2
30   }.
31 End FuzzyRelationLayer.

```

A base similarity relation between symbols, $\mathcal{R} : \Sigma \times \Sigma \rightarrow [0, 1]$, induces similarities between words and between languages.

Definition 3.13 (Similarity on words). *Given a similarity relation $\mathcal{R}$ on Σ , define a proximity relation on words $\mathcal{R}^w : \Sigma^* \times \Sigma^* \rightarrow [0, 1]$ by*

$$\mathcal{R}^w(w_1, w_2) = \begin{cases} \bigotimes_{1 \leq i \leq n} \mathcal{R}(a_i, b_i) & \text{if } w_1 = a_1 \cdots a_n \text{ and } w_2 = b_1 \cdots b_n; \\ 0 & \text{otherwise.} \end{cases}$$

Example. With the base relation $\mathcal{R}$ of the previous example and the minimum T-norm, $\mathcal{R}^w(ab, cd) = \min(\mathcal{R}(a, c), \mathcal{R}(b, d)) = \min(0.9, 0.75) = 0.75$, while $\mathcal{R}^w(ab, cb) = \min(\mathcal{R}(a, c), \mathcal{R}(b, b)) = \min(0.9, 1) = 0.9$. The words ab and a have different lengths, so $\mathcal{R}^w(ab, a) = 0$.

Listing 20: Similarity on words (SimilarityRelations.v).

```

1 Fixpoint Rw_T (T : T_norm) (w1 w2 : word) : simval :=
2   match w1, w2 with
3   | [::], [::] => 1
4   | [::], _   => 0
5   | _ , [::] => 0
6   | a1 :: t1, a2 :: t2 =>
7     tnorm T (R a1 a2) (Rw_T T t1 t2)
8   end.

```

Definition 3.14 (Similarity on languages). *For languages $L_1, L_2 \subseteq \Sigma^*$ define a proximity relation $\mathcal{R}^L : \mathcal{P}(\Sigma^*) \times \mathcal{P}(\Sigma^*) \rightarrow [0, 1]$ by*

$$\mathcal{R}^L(L_1, L_2) = \min \left\{ \inf_{w_1 \in L_1} \sup_{w_2 \in L_2} \mathcal{R}^w(w_1, w_2), \inf_{w_2 \in L_2} \sup_{w_1 \in L_1} \mathcal{R}^w(w_1, w_2) \right\}.$$

Example. Continuing with the same $\mathcal{R}$ and the minimum, compare $L_1 = \{ab\}$ and $L_2 = \{cd, cb\}$. In the direction from L_1 , the single word ab has $\sup\{\mathcal{R}^w(ab, cd), \mathcal{R}^w(ab, cb)\} = \max(0.75, 0.9) = 0.9$, so the infimum over L_1 is 0.9. In the direction from L_2 , the word cd gives $\sup\{\mathcal{R}^w(ab, cd)\} = 0.75$ and cb gives $\sup\{\mathcal{R}^w(ab, cb)\} = 0.9$, so the infimum over L_2 is $\min(0.75, 0.9) = 0.75$. The language similarity is the minimum of the two directions, $\mathcal{R}^L(L_1, L_2) = \min(0.9, 0.75) = 0.75$.

Listing 21: Similarity on languages (SimilarityRelations.v).

```

1 Definition RL_T (L1 L2 : language) : simval :=
2   smin
3     (@Infl L1
4       (fun w1 =>
5         @SupL L2 (fun w2 => Rw_T T w1 w2)
6           (Rw_T_upper_bounded_right w1 L2))
7         (outer_T_lower_bounded_left L1 L2))
8     (@Infl L2
9       (fun w2 =>
10        @SupL L1 (fun w1 => Rw_T T w1 w2)
11          (Rw_T_upper_bounded_left w2 L1))
12        (outer_T_lower_bounded_right L1 L2)).

```

Viewing words as regular expressions, Definition 3.13 extends to regular expressions when Gödel's T-norm is used.

Definition 3.15 (Similarity on regular expressions, Gödel's T-norm). *Given a similarity relation $\mathcal{R}$ on the alphabet, define a proximity on regular expressions $\mathcal{G}^E : E^\Sigma \times E^\Sigma \rightarrow [0, 1]$ by*

$$\mathcal{G}^E(r_1, r_2) = \begin{cases} \bigotimes_{1 \leq i \leq n} \mathcal{R}(a_i, b_i) & \text{if } r_1 = r[a_1, \dots, a_n] \text{ and } r_2 = r[b_1, \dots, b_n]; \\ 0 & \text{otherwise.} \end{cases}$$

In Rocq this is the recursive function GE. It recurses on the shape of the two expressions, returns the base similarity R on letters, the minimum on matching binary shapes, and 0 when the shapes differ.

Listing 22: Gödel similarity on regular expressions (SimilarityRelations.v).

```

1 Fixpoint GE (r s : regex) : simval :=
2   match s with
3   | RS.Empty =>
4     match r with
5     | RS.Empty => 1
6     | _ => 0
7     end
8   | RS.Eps =>
9     match r with
10    | RS.Eps => 1
11    | _ => 0
12    end
13  | RS.Char a =>
14    match r with
15    | RS.Char b => R b a

```

```

16     | _ => 0
17     end
18 | RS.Star s1 =>
19     match r with
20     | RS.Star r1 => GE r1 s1
21     | _ => 0
22     end
23 | RS.Seq s1 s2 =>
24     match r with
25     | RS.Seq r1 r2 => smin (GE r1 s1) (GE r2 s2)
26     | _ => 0
27     end
28 | RS.Alt s1 s2 =>
29     match r with
30     | RS.Alt r1 r2 => smin (GE r1 s1) (GE r2 s2)
31     | _ => 0
32     end
33 end.

```

For arbitrary T-norms Definition 3.15 is too generous over regular expressions. If $\mathcal{R}(a, b) \leq 1$ then $\mathcal{R}^L(\mathcal{L}(a^*), \mathcal{L}(b^*)) = \mathcal{R}(a, b)$ only when $\mathcal{R}(a, b)$ is idempotent; but a T-norm all of whose values are idempotent is necessarily Gödel's minimum. There are nonetheless many non-idempotent T-norms one wishes to consider, for example

$$T(x, y) = \begin{cases} p + \frac{(x-p)(y-p)}{1-p} & x, y \geq p; \\ \min(x, y) & \text{otherwise,} \end{cases}$$

where $p \in [0, 1]$. Definition 3.15 ergo has to be conservatively extended.

Definition 3.16 (Similarity on regular expressions, general T-norm). *Let r, s be restricted regular expressions over Σ and $\mathcal{R}$ a similarity on Σ . Define inductively the proximity relation*

$$\mathcal{R}^e(r, s) = \begin{cases} 1 & \text{if } r = s; \\ \mathcal{R}(a, b) & \text{if } r = a \text{ and } s = b; \\ \min\{\mathcal{R}^e(r_1, s_1), \mathcal{R}^e(r_2, s_2)\} & \text{if } r = r_1 + r_2 \text{ and } s = s_1 + s_2; \\ \mathcal{R}^e(r_1, s_1) \otimes \mathcal{R}^e(r_2, s_2) & \text{if } r = r_1 \cdot r_2 \text{ and } s = s_1 \cdot s_2; \\ \inf_n (\bigotimes^n \mathcal{R}^e(r_1, s_1)) & \text{if } r = r_1^* \text{ and } s = s_1^*; \\ 0 & \text{otherwise.} \end{cases}$$

The clause for $()^*$ is strict for a general T-norm, but no definition for a general T-norm seems reasonable unless it is replaced by a finite union. In Rocq this is the recursive function `RE_T`, parameterised by the T-norm `T`. The union and concatenation clauses both combine the subexpression values with `tnorm T`, and the star clause uses `tnorm_power_inf T`, the infimum of the T-norm powers $\inf_n (\bigotimes^n x)$. Under Gödel's minimum, which is idempotent, `tnorm_power_inf` collapses to the identity and `RE_T` reduces to `GE`.

Listing 23: Syntactic similarity on expressions, general T-norm (`SimilarityRelations.v`).

```

1 Fixpoint RE_T (r s : regex) {struct s} : simval :=
2   match s with
3   | RS.Empty =>
4     match r with
5     | RS.Empty => 1
6     | _ => 0
7     end
8   | RS.Eps =>
9     match r with
10    | RS.Eps => 1
11    | _ => 0
12    end
13  | RS.Char b =>
14    match r with
15    | RS.Char a => R a b
16    | _ => 0
17    end
18  | RS.Alt s1 s2 =>
19    match r with
20    | RS.Alt r1 r2 => tnorm T (RE_T r1 s1) (RE_T r2 s2)
21    | _ => 0
22    end
23  | RS.Seq s1 s2 =>
24    match r with
25    | RS.Seq r1 r2 => tnorm T (RE_T r1 s1) (RE_T r2 s2)
26    | _ => 0
27    end
28  | RS.Star s1 =>
29    match r with
30    | RS.Star r1 => tnorm_power_inf T (RE_T r1 s1)
31    | _ => 0
32    end
33 end.

```

Example. With the same base relation and the minimum, comparing expressions of matching shape gives $\mathcal{R}^e(a \cdot b, c \cdot d) = \min(\mathcal{R}(a, c), \mathcal{R}(b, d)) = \min(0.9, 0.75) = 0.75$ and $\mathcal{R}^e(a \cdot b, c \cdot b) = \min(0.9, 1) = 0.9$, while a star gives $\mathcal{R}^e(a^*, c^*) = \mathcal{R}(a, c) = 0.9$ because the minimum is idempotent. When the shapes differ the value is 0, for instance $\mathcal{R}^e(a \cdot b, c + d) = 0$ since one expression is a concatenation and the other a union.

The three constructions are all similarity relations.

Lemma 3.3.

1. $\mathcal{R}^e$ is a similarity relation on E^Σ , and therefore on restricted regular expressions;
2. $\mathcal{R}^w$ is a similarity relation on Σ^* ;
3. $\mathcal{R}^L$ is a similarity relation on languages.

Proof. All three are formalised, and bundled together in the lemma `general_similarity_relations` of Listing 24. Fix any T-norm T that is left-continuous and with respect to which the base relation R is transitive. Then $\mathcal{R}^e$, $\mathcal{R}^w$ and $\mathcal{R}^L$ are all T -similarity relations. Each is proved by induction. For words the induction is on the length, and transitivity follows because the head and the tail can be combined through the monotonicity of the T-norm. For expressions the induction is on the shape that the two share. For languages the result is read off from the word case, using that a language similarity is built from suprema and infima of word similarities. $\square$

Listing 24: The three similarity relations, formalised (`SimilarityRelations.v`).

```

1 (* Similarity relations on restricted regular expressions, words, and languages. *)
2 Lemma general_similarity_relations :
3   forall T,
4     left_continuous_on_01 (tnorm T) ->
5     (forall a b c, tnorm T (R a b) (R b c) <= R a c) ->
6     @is_T_similarity restricted_regex_type T (RE_T_restricted T) /\
7     @is_T_similarity word T (Rw_T T) /\
8     @is_T_similarity language T (RL_T T).
9 (* Proof in SimilarityRelations.v of the repository [10]. *)

```

Remark 3.4.

1. Definition 3.16 is syntax-sensitive. Two regular expressions may denote the same language while having low, or even zero, $\mathcal{R}^e$ -similarity. For example, depending on the base relation, $\mathcal{R}^e(a + b, b + a) = 0$ even though $\mathcal{L}(a + b) = \mathcal{L}(b + a)$.

2. For $r_1, r_2 \in R^\Sigma$ and words $w_1 \in \mathcal{L}(r_1)$, $w_2 \in \mathcal{L}(r_2)$ of the same length, $\mathcal{R}^e(r_1, r_2)$ and $\mathcal{R}^w(w_1, w_2)$ are in general incomparable: $\mathcal{R}^w(a, a) = 1$ but $\mathcal{R}^e(a, a + a) = 0$, while $\mathcal{R}^w(a, b) = 0$ but $\mathcal{R}^e(a + b, a + b) = 1$.
3. One may build finite equational properties into the definition, for instance setting $\mathcal{R}^e(r_1, s) = \mathcal{R}^e(r_2, s)$ whenever $\text{nf}(r_1) = \text{nf}(r_2)$, but no exhaustive finitary definition is possible, and this is not pursued here.
4. The bound $\mathcal{R}^e(r_1, r_2) \leq \mathcal{R}^L(\mathcal{L}(r_1), \mathcal{L}(r_2))$ fails once $\&$ and $\neg$ are allowed. If $\mathcal{R}(a, b) \geq 0$ but $a \neq b$, then $\mathcal{L}(a \& b) = \emptyset$, so $\mathcal{R}^L(\mathcal{L}(a \& a), \mathcal{L}(a \& b)) = 0$, whereas a shape-based value $\mathcal{R}^e(a \& a, a \& b) = \mathcal{R}(a, b) > 0$. This is why the fuzzy theory is confined to the restricted fragment R^Σ .

Lemma 3.4 (Finite syntactic similarity). *If Σ is finite, then for every restricted regular expression r the set $\{s \mid \mathcal{R}^e(s, r) \neq 0\}$ is finite.*

Proof. This is formalised as the lemma `finite_syntactic_similarity` of Listing 25. The development builds a finite list `syntactic_similarity_candidates` `alphabet r`, which enumerates all expressions sharing the outermost shape of r with letters drawn from the alphabet, and proves, by induction on the derivation that r is restricted, that every s with $\mathcal{R}^e(s, r) \neq 0$ (that is, $\text{RE_T } T \ s \ r \neq 0$) belongs to that list. The base cases use that for letters only the finitely many b with $\mathcal{R}(b, a) \neq 0$ qualify, and the inductive cases use that the T-norm is non-zero only when both arguments are. Since the candidate list is finite, so is $\{s \mid \mathcal{R}^e(s, r) \neq 0\}$. $\square$

Listing 25: Finiteness of syntactic similarity, formalised (`SimilarityRelations.v`).

```

1 Fixpoint syntactic_similarity_candidates
2   (alphabet : list A) (r : regex) : list regex :=
3   match r with
4   | RS.Empty => (RS.Empty :: nil)%list
5   | RS.Eps => (RS.Eps :: nil)%list
6   | RS.Char _ => List.map RS.Char alphabet
7   | RS.Alt r1 r2 =>
8     List.flat_map
9       (fun s1 =>
10        List.map (fun s2 => RS.Alt s1 s2)
11          (syntactic_similarity_candidates alphabet r2))
12        (syntactic_similarity_candidates alphabet r1)
13   | RS.Seq r1 r2 =>
14     List.flat_map
15       (fun s1 =>

```

```

16         List.map (fun s2 => RS.Seq s1 s2)
17             (syntactic_similarity_candidates alphabet r2))
18     (syntactic_similarity_candidates alphabet r1)
19 | RS.Star r1 =>
20     List.map RS.Star
21         (syntactic_similarity_candidates alphabet r1)
22 end.
23
24 (* Finite Syntactic Similarity *)
25 Lemma finite_syntactic_similarity :
26     forall T alphabet r,
27         (forall a, List.In a alphabet) ->
28         RS.restricted_regex r ->
29         forall s,
30             RE_T T s r <> 0 ->
31             List.In s (syntactic_similarity_candidates alphabet r).
32 (* Proof in SimilarityRelations.v of the repository [10]. *)

```

3.5 Fuzzy Derivatives and Fuzzy Automata

The derivative now carries over to the fuzzy setting. Comparing only the consumed symbol with the first symbol of a word in the language would be too narrow. In a similarity-based setting the remaining suffix should also be matched up to the same cut, so the whole word is matched. Formalising the fuzzy derivative and the fuzzy automaton in Rocq is ongoing research.

Definition 3.17 (Fuzzy language derivative). *Let $\mathcal{L} \subseteq \Sigma^*$, $a \in \Sigma$ and $0 < \mu \leq 1$. The μ -fuzzy derivative of $\mathcal{L}$ with respect to a is*

$$\partial_a^\mu(\mathcal{L}) = \{ w \in \Sigma^* \mid \exists b \in \Sigma, \exists v \in \Sigma^* \text{ such that } \mathcal{R}(a, b) \otimes \mathcal{R}^w(w, v) \geq \mu \text{ and } bv \in \mathcal{L} \}.$$

Definition 3.18 (Fuzzy expression derivative). *Let r be a restricted regular expression over Σ , $a \in \Sigma$ and $0 < \mu \leq 1$. The fuzzy derivative of r with respect to a and μ is*

$$\mathcal{D}_a^\mu(r) = \sum_{\mathcal{R}(a, b) \otimes \mathcal{R}^e(s, r) \geq \mu} \mathcal{D}_b(s),$$

where $\mathcal{D}_b(s)$ is the ordinary Brzozowski derivative of Section 3.2.

Example. With $\mathcal{R}(a, c) = 0.9$, the other values as in the running example, $\mu = 0.7$ and $r = a$, the symbols μ -similar to a are a and c , and the expressions μ -similar to a are likewise a and c . The sum

therefore ranges over $b, s \in \{a, c\}$ and gives

$$\mathcal{D}_a^\mu(a) = \mathcal{D}_a(a) + \mathcal{D}_c(c) + \mathcal{D}_c(a) + \mathcal{D}_a(c) = \varepsilon + \varepsilon + \emptyset + \emptyset,$$

which canonicalises to ε .

Definition 3.19 (Fuzzy derivative with respect to a word). *For a word $u = av$ with $a \in \Sigma$ and $v \in \Sigma^*$, the fuzzy derivatives of a language and of a restricted regular expression with respect to u are*

$$\partial_u^\mu(\mathcal{L}) = \bigcup_{\mu' \otimes \mu'' \geq \mu} \partial_v^{\mu'}(\partial_a^{\mu''}(\mathcal{L})), \quad \mathcal{D}_u^\mu(r) = \sum_{\mu' \otimes \mu'' \geq \mu} \mathcal{D}_v^{\mu'}(\mathcal{D}_a^{\mu''}(r)),$$

with $\mathcal{D}_\varepsilon^\mu(r) = r$.

Both expression derivatives are syntactic. By Lemma 3.4 the summands in the sums are finite, so $\mathcal{D}_u^\mu(r)$ is a well-defined restricted regular expression.

From now on $\otimes$ is Gödel's minimum T-norm. Its idempotence makes the cut relations equivalences, which is what reduces the fuzzy theory to the crisp one.

Proposition 3.1 (Cut equivalences). *Let $\mathcal{R}$ be a fuzzy similarity relation on Σ and $0 < \mu \leq 1$. Each of the relations*

$$\begin{aligned} a \sim_\mu^\Sigma b &\iff \mathcal{R}(a, b) \geq \mu, & r \sim_\mu^e s &\iff \mathcal{R}^e(r, s) \geq \mu, \\ w_1 \sim_\mu^{\Sigma^*} w_2 &\iff \mathcal{R}^w(w_1, w_2) \geq \mu, & L_1 \sim_\mu^{\mathcal{P}(\Sigma^*)} L_2 &\iff \mathcal{R}^L(L_1, L_2) \geq \mu, \end{aligned}$$

on symbols, restricted expressions, words and languages respectively, is an equivalence relation. Moreover $a_1 \cdots a_n \sim_\mu^{\Sigma^} b_1 \cdots b_m$ if and only if $n = m$ and $a_i \sim_\mu^\Sigma b_i$ for every i .*

Under the minimum, the condition $\mathcal{R}(a, b) \otimes \mathcal{R}^w(w, v) \geq \mu$ splits into $\mathcal{R}(a, b) \geq \mu$ and $\mathcal{R}^w(w, v) \geq \mu$, that is $(a, b) \in \mathcal{R}_\mu$ and $w \sim_\mu^{\Sigma^*} v$, so the fuzzy language derivative reads

$$\partial_a^\mu(\mathcal{L}) = \{ w \mid \exists b, v : (a, b) \in \mathcal{R}_\mu, w \sim_\mu^{\Sigma^*} v, bv \in \mathcal{L} \}.$$

Definition 3.20 (Cut neighbourhood). *For $a \in \Sigma$ and $0 < \mu \leq 1$, the cut neighbourhood of a is $\mathcal{S}_a^\mu = \{ b \in \Sigma \mid (a, b) \in \mathcal{R}_\mu \}$, that is, the μ -equivalence class of a .*

Definition 3.21 (Quotient alphabet and language). *Let $\Sigma^\mu := \Sigma / \sim_\mu^\Sigma$ and let $q_\mu : \Sigma \rightarrow \Sigma^\mu$, $q_\mu(a) = [a]_\mu$, be the quotient map, extended to words letterwise by $q_\mu^*(a_1 \cdots a_n) = [a_1]_\mu \cdots [a_n]_\mu$. For a language $\mathcal{L} \subseteq \Sigma^*$, its quotient image is $\mathcal{L}^\mu := q_\mu^*(\mathcal{L}) = \{ q_\mu^*(w) \mid w \in \mathcal{L} \} \subseteq (\Sigma^\mu)^*$.*

Once letters are identified up to μ -similarity, the fuzzy derivative becomes an ordinary derivative over the quotient alphabet.

Theorem 3.5 (Quotient form of the fuzzy derivative). *For every language $\mathcal{L} \subseteq \Sigma^*$ and every $a \in \Sigma$,*

$$\partial_a^\mu(\mathcal{L}) = \{ w \in \Sigma^* \mid q_\mu^*(w) \in \partial_{[a]_\mu}(\mathcal{L}^\mu) \}, \quad \text{ergo} \quad q_\mu^*(\partial_a^\mu(\mathcal{L})) = \partial_{[a]_\mu}(\mathcal{L}^\mu).$$

Proof. The two inclusions are proved as follows. Let $w \in \partial_a^\mu(\mathcal{L})$. There are $b \in \Sigma$ and $v \in \Sigma^*$ with $\mathcal{R}(a, b) \geq \mu$, $w \sim_\mu^{\Sigma^*} v$ and $bv \in \mathcal{L}$. Then $[b]_\mu = [a]_\mu$ and $q_\mu^*(w) = q_\mu^*(v)$, so

$$[a]_\mu q_\mu^*(w) = [b]_\mu q_\mu^*(v) = q_\mu^*(bv) \in \mathcal{L}^\mu,$$

whence $q_\mu^*(w) \in \partial_{[a]_\mu}(\mathcal{L}^\mu)$. Conversely, suppose $q_\mu^*(w) \in \partial_{[a]_\mu}(\mathcal{L}^\mu)$. Then $[a]_\mu q_\mu^*(w) \in \mathcal{L}^\mu$, so there is a word $bv \in \mathcal{L}$ with $q_\mu^*(bv) = [a]_\mu q_\mu^*(w)$. Comparing first letters gives $[b]_\mu = [a]_\mu$, i.e. $\mathcal{R}(a, b) \geq \mu$, and comparing the remainders gives $q_\mu^*(v) = q_\mu^*(w)$, i.e. $w \sim_\mu^{\Sigma^*} v$; thus $w \in \partial_a^\mu(\mathcal{L})$. The displayed consequence follows because $\partial_{[a]_\mu}(\mathcal{L}^\mu)$ consists only of quotient words, and each such word has at least one representative in Σ^* . $\square$

Example. Let $\Sigma = \{a, b, c, d\}$ and let $\mathcal{R}$ be the similarity relation with $\mathcal{R}(x, x) = 1$ for every x , $\mathcal{R}(a, c) = \mathcal{R}(c, a) = 0.9$, $\mathcal{R}(b, d) = \mathcal{R}(d, b) = 0.75$, and $\mathcal{R}(x, y) = 0$ for all other distinct x, y . Take the cut $\mu = 0.7$. The μ -cut is

$$\mathcal{R}_\mu = \{(a, a), (b, b), (c, c), (d, d), (a, c), (c, a), (b, d), (d, b)\},$$

so its classes are $[a]_\mu = \{a, c\}$ and $[b]_\mu = \{b, d\}$, and the quotient alphabet is $\Sigma^\mu = \{[a]_\mu, [b]_\mu\}$.

Consider $\mathcal{L} = \{ab, cd, cb\}$. To compute $\partial_a^\mu(\mathcal{L})$, collect the words of $\mathcal{L}$ whose first letter is μ -similar to a , that is, lies in $\{a, c\}$, and keep their suffixes up to μ -similarity. All three words ab, cd, cb begin with a letter in $\{a, c\}$, leaving the suffixes b, d, b , each of which is μ -similar exactly to the words in $\{b, d\}$. Ergo

$$\partial_a^\mu(\mathcal{L}) = \{b, d\} = \partial_c^\mu(\mathcal{L}),$$

the two being equal because $a \sim_\mu^{\Sigma} c$. No word of $\mathcal{L}$ begins with a letter in the class $\{b, d\}$, so $\partial_b^\mu(\mathcal{L}) = \partial_d^\mu(\mathcal{L}) = \emptyset$.

The quotient construction gives the same answer directly. Applying q_μ^* letterwise, $q_\mu^*(ab) = q_\mu^*(cd) = q_\mu^*(cb) = [a]_\mu[b]_\mu$, so $\mathcal{L}^\mu = \{[a]_\mu[b]_\mu\}$. Its ordinary derivative with respect to $[a]_\mu$ is $\partial_{[a]_\mu}(\mathcal{L}^\mu) = \{[b]_\mu\}$, and taking the inverse image under q_μ^* recovers $(q_\mu^*)^{-1}(\{[b]_\mu\}) = \{b, d\} = \partial_a^\mu(\mathcal{L})$, exactly as Theorem 3.5 predicts.

The quotient viewpoint immediately yields closure under derivation.

Theorem 3.6 (Regularity of fuzzy derivatives). *For every $u \in \Sigma^*$ and every $0 < \mu \leq 1$, if $\mathcal{L}$ is regular then $\partial_u^\mu(\mathcal{L})$ is regular; and if r is restricted then $\mathcal{D}_u^\mu(r)$ is a restricted regular expression.*

Proof. Fix $0 < \mu \leq 1$ and let $\mathcal{L}$ be regular. For a single symbol a , Theorem 3.5 gives $\partial_a^\mu(\mathcal{L}) = (q_\mu^*)^{-1}(\partial_{[a]_\mu}(\mathcal{L}^\mu))$. Since q_μ^* is a homomorphism and $\mathcal{L}$ is regular, the quotient image $\mathcal{L}^\mu$ is regular over the finite alphabet Σ^μ ; classical derivatives preserve regularity (Theorem 3.2), so $\partial_{[a]_\mu}(\mathcal{L}^\mu)$ is regular; and the inverse homomorphic image of a regular language is regular. Thus $\partial_a^\mu(\mathcal{L})$ is regular. For a word $u = a_1 \cdots a_n$ the same argument applies at each step of $\partial_u^\mu(\mathcal{L}) = \partial_{a_n}^\mu(\cdots \partial_{a_1}^\mu(\mathcal{L}) \cdots)$, so $\partial_u^\mu(\mathcal{L})$ is regular. The expression statement is immediate from Definitions 3.18 and 3.19. Ordinary Brzozowski derivatives are restricted regular expressions, and a finite sum of restricted regular expressions is restricted. $\square$

To relate the syntactic and semantic sides, an expression is interpreted up to μ -similarity.

Definition 3.22 (Fuzzy semantic interpretation). *For a restricted regular expression r and $0 < \mu \leq 1$, the fuzzy semantic interpretation of r is the language*

$$\mathcal{L}^\mu(r) = \{ w \in \Sigma^* \mid w \sim_\mu^{\Sigma^*} v \text{ for some } v \in \mathcal{L}(r) \}.$$

Example. With the running data and $r = a \cdot b$, the language is $\mathcal{L}(r) = \{ab\}$, and the words μ -similar to ab are exactly those xy with $x \in \{a, c\}$ and $y \in \{b, d\}$, so $\mathcal{L}^\mu(a \cdot b) = \{ab, ad, cb, cd\}$.

Theorem 3.7 (Semantic–syntactic correspondence). *Let r be a restricted regular expression over Σ , $u \in \Sigma^*$ and $0 < \mu \leq 1$. Then $\mathcal{L}^\mu(\mathcal{D}_u^\mu(r)) = \partial_u^\mu(\mathcal{L}(r))$.*

Proof. First treat a single symbol a . By Definition 3.18 and the distributivity of the semantics over $+$,

$$\mathcal{L}(\mathcal{D}_a^\mu(r)) = \bigcup_{a \sim_\mu^\Sigma b, s \sim_\mu^e r} \mathcal{L}(\mathcal{D}_b(s)) = \bigcup_{a \sim_\mu^\Sigma b, s \sim_\mu^e r} \partial_b(\mathcal{L}(s)),$$

where the second equality is the crisp semantic–syntactic correspondence (Theorem 3.1). By Definition 3.16 the relation $s \sim_\mu^e r$ holds exactly when s has the same shape as r with each letter replaced by a μ -similar one, so as b ranges over the class of a and s over the class of r , the words bv with $v \in \mathcal{L}(s)$ range exactly over the words that are μ -similar, letter by letter, to words of $\mathcal{L}(r)$ and begin with a letter μ -similar to a . Closing the displayed union under μ -similarity of the suffix ergo yields precisely the set described in Definition 3.17, that is $\mathcal{L}^\mu(\mathcal{D}_a^\mu(r)) = \partial_a^\mu(\mathcal{L}(r))$. For a word $u = av$ the argument is by induction on $|u|$. Using Definitions 3.19 and the single-symbol case,

$$\mathcal{L}^\mu(\mathcal{D}_u^\mu(r)) = \partial_v^\mu(\partial_a^\mu(\mathcal{L}(r))) = \partial_u^\mu(\mathcal{L}(r)),$$

which completes the proof. $\square$

A deterministic automaton for the fuzzy language can now be built exactly as in the crisp case,

taking canonical fuzzy derivatives as states. The first step is to see that there are only finitely many of them.

Theorem 3.8 (Finiteness of fuzzy language derivatives). *Let Σ be finite and $0 < \mu \leq 1$. If $\mathcal{L} \subseteq \Sigma^*$ is regular, then the set $\{ \partial_u^\mu(\mathcal{L}) \mid u \in \Sigma^* \}$ is finite.*

Proof. Since q_μ^* is a homomorphism, the quotient image $\mathcal{L}^\mu$ is a regular language over the finite alphabet Σ^μ . By Theorem 3.3 the set $D^\mu = \{ \partial_x(\mathcal{L}^\mu) \mid x \in (\Sigma^\mu)^* \}$ is finite. By repeated application of Theorem 3.5, every fuzzy derivative has the form $\partial_u^\mu(\mathcal{L}) = (q_\mu^*)^{-1}(\partial_{q_\mu^*(u)}(\mathcal{L}^\mu))$, the inverse image of an element of the finite set D^μ . Ergo $\{ \partial_u^\mu(\mathcal{L}) \mid u \in \Sigma^* \}$ is finite. $\square$

Theorem 3.9 (Finiteness of fuzzy expression derivatives). *Let Σ be finite and $0 < \mu \leq 1$. For a restricted regular expression R , the set $\{ \mathcal{L}^\mu(\mathcal{D}_w^\mu(R)) \mid w \in \Sigma^* \}$ is finite.*

Proof. By the semantic–syntactic correspondence (Theorem 3.7), $\mathcal{L}^\mu(\mathcal{D}_w^\mu(R)) = \partial_w^\mu(\mathcal{L}(R))$ for every $w \in \Sigma^*$, so

$$\{ \mathcal{L}^\mu(\mathcal{D}_w^\mu(R)) \mid w \in \Sigma^* \} = \{ \partial_w^\mu(\mathcal{L}(R)) \mid w \in \Sigma^* \},$$

and the right-hand set is finite by Theorem 3.8. $\square$

Finiteness guarantees that the construction of Section 3.3, run with fuzzy derivatives, terminates. The automaton recognises the words that lie within tolerance μ of the language of r .

Theorem 3.10 (Fuzzy DFA correctness). *Let r be a restricted regular expression over Σ , let $0 < \mu \leq 1$, and let $M^\mu = \langle Q^\mu, \Sigma, \delta^\mu, q_0^\mu, F^\mu \rangle$ be the DFA constructed from r with fuzzy derivatives at cut μ , that is, with $q_0^\mu = [r]$, $\delta^\mu(q, a) = [\mathcal{D}_a^\mu(q)]$, and $F^\mu = \{q \mid \nu(q) = \varepsilon\}$. Writing*

$$\mathcal{L}_\Sigma^\mu(r) = (q_\mu^*)^{-1}(q_\mu^*(\mathcal{L}(r)))$$

for the words that are μ -similar to some word of r , the automaton satisfies

$$\mathcal{L}(M^\mu) = \mathcal{L}_\Sigma^\mu(r).$$

Proof. Let $\delta^{\mu*}$ extend δ^μ to words by $\delta^{\mu*}(q, \varepsilon) = q$ and $\delta^{\mu*}(q, wa) = \delta^\mu(\delta^{\mu*}(q, w), a)$. Induction on w shows $\delta^{\mu*}(q_0^\mu, w) = [\mathcal{D}_w^\mu(r)]$. For $w = \varepsilon$ this is $q_0^\mu = [r] = [\mathcal{D}_\varepsilon^\mu(r)]$. For $w = ua$, the induction hypothesis and the definition of δ^μ give

$$\delta^{\mu*}(q_0^\mu, ua) = \delta^\mu([\mathcal{D}_u^\mu(r)], a) = [\mathcal{D}_a^\mu(\mathcal{D}_u^\mu(r))] = [\mathcal{D}_{ua}^\mu(r)].$$

Therefore, for any $w \in \Sigma^*$,

$$\begin{aligned} w \text{ accepted by } M^\mu &\iff \delta^{\mu*}(q_0^\mu, w) \in F^\mu \iff \nu(\mathcal{D}_w^\mu(r)) = \varepsilon \iff \varepsilon \in \mathcal{L}(\mathcal{D}_w^\mu(r)) \\ &\iff \varepsilon \in \mathcal{L}^\mu(\mathcal{D}_w^\mu(r)) \iff \varepsilon \in \partial_w^\mu(\mathcal{L}(r)) \iff w \in \mathcal{L}_\Sigma^\mu(r), \end{aligned}$$

where the fourth step uses that the only word μ -similar to ε is ε itself, the fifth is Theorem 3.7, and the last is the definition of $\mathcal{L}_\Sigma^\mu(r)$ together with the quotient form of the fuzzy derivative. Ergo $\mathcal{L}(M^\mu) = \mathcal{L}_\Sigma^\mu(r)$. $\square$

4 Conclusion

Two questions stand behind this thesis. The first is whether the derivative-based theory of regular languages still works once exact symbol matching is dropped and symbols match by similarity instead. The second is whether the core of that theory can be checked by a proof assistant rather than trusted on paper. The preceding sections answer yes to both.

On the crisp side the thesis develops derivatives of regular languages and of regular expressions. Everything else rests on Theorem 3.1. It says that the syntactic derivative of an expression and the semantic derivative of its language describe the same thing once an expression is read as the language it denotes. From there derivatives stay regular (Theorem 3.2) and an expression has only finitely many derivative languages (Theorem 3.3). That finiteness makes the derivative automaton possible. Taking canonical derivatives as states and differentiation as the transition yields a deterministic automaton that recognises the right language (Theorem 3.4) and that a terminating algorithm builds.

In the fuzzy layer equality of symbols is replaced by a fuzzy similarity relation together with a cut value μ . Fuzzy derivatives of both languages and restricted regular expressions arise (Definitions 3.17 and 3.18). The idea that makes the layer work is simple. Under the minimum T-norm, cut-similarity is an equivalence relation (Proposition 3.1). Once that is in hand, a fuzzy derivative over Σ is just an ordinary derivative over the quotient alphabet Σ^μ obtained by collapsing μ -similar symbols into a single class (Theorem 3.5). This quotient picture lets the crisp theory carry over, so regularity (Theorem 3.6), the correspondence between syntax and semantics (Theorem 3.7), finiteness (Theorem 3.8) and fuzzy DFA correctness (Theorem 3.10) all follow. The fuzzy automaton accepts the words that come within threshold μ of some word in the language.

The crisp theory is formalised in Rocq. It covers the syntax and semantics, nullability, canonicalisation and its correctness, the derivative correspondence and DFA correctness, and it turns these claims into statements a proof assistant has checked. The similarity infrastructure that the fuzzy results build on is formalised there as well. The next step is to settle the Kuratowski-finiteness result for canonical derivatives. After that the fuzzy layer can be implemented.

Bibliography

- [1] S. Alves, D. Kesner, and M. Ramos, “Extending the quantitative pattern-matching paradigm,” in *Programming Languages and Systems (APLAS)*, Singapore: Springer Nature Singapore, 2025, pp. 84–105.
- [2] Y. Bertot and P. Castéran, *Interactive Theorem Proving and Program Development. Coq 'Art: The Calculus of Inductive Constructions* (Texts in Theoretical Computer Science). Springer, 2004. DOI: 10.1007/978-3-662-07964-5
- [3] J. A. Brzozowski, “Derivatives of regular expressions,” *Journal of the ACM*, vol. 11, no. 4, pp. 481–494, 1964. DOI: 10.1145/321239.321249
- [4] J. H. Conway, *Regular Algebra and Finite Machines*. London: Chapman and Hall, 1971.
- [5] T. Coquand and G. Huet, “The calculus of constructions,” *Information and Computation*, vol. 76, no. 2–3, pp. 95–120, 1988. DOI: 10.1016/0890-5401(88)90005-3
- [6] T. Coquand and V. Siles, “A decision procedure for regular expression equivalence in type theory,” in *Proceedings of the 1st International Conference on Certified Programs and Proofs (CPP)*, Springer, 2011, pp. 119–134. DOI: 10.1007/978-3-642-25379-9_11
- [7] S. Garhwal, R. Jiware, and S. Tomasiello, “Fuzzy regular languages and fuzzy automata,” in *2017 International Conference on High Performance Computing and Simulation (HPCS)*, Genoa: IEEE, 2017, pp. 477–482. DOI: 10.1109/HPCS.2017.77
- [8] V. M. Glushkov, “The abstract theory of automata,” *Russian Mathematical Surveys*, vol. 16, no. 5, pp. 1–53, 1962, ISSN: 0036-0279. DOI: 10.1070/rm1961v016n05ABEH004112
- [9] W. A. Howard, “The formulae-as-types notion of construction,” in *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, J. P. Seldin and J. R. Hindley, Eds., Manuscript circulated from 1969, Academic Press, 1980, pp. 479–490.
- [10] M. Katamashvili and D. Maisuradze, *Rocq formalization for regular-expression derivatives with similarity*, <https://github.com/DUT1K1/regular-expression-derivative>, 2026.
- [11] S. C. Kleene, “Representation of events in nerve nets and finite automata,” in *Automata Studies*, ser. Annals of Mathematics Studies 34, C. E. Shannon and J. McCarthy, Eds., Princeton, NJ: Princeton University Press, 1956, pp. 3–41. DOI: 10.1515/9781400882618-002
- [12] D. Kozen, “A completeness theorem for kleene algebras and the algebra of regular events,” *Information and Computation*, vol. 110, no. 2, pp. 366–390, 1994. DOI: 10.1006/inco.1994.1037

- [13] Y. Li and W. Pedrycz, “Fuzzy finite automata and fuzzy regular expressions with membership values in lattice-ordered monoids,” *Fuzzy Sets and Systems*, vol. 156, no. 1, pp. 68–92, 2005. DOI: 10.1016/j.fss.2005.04.004
- [14] P. Martin-Löf, *Intuitionistic Type Theory*. Naples: Bibliopolis, 1984, Notes by Giovanni Sambin.
- [15] R. McNaughton and H. Yamada, “Regular expressions and state graphs for automata,” *IRE Transactions on Electronic Computers*, vol. EC-9, no. 1, pp. 39–47, 1960. DOI: 10.1109/TEC.1960.5221603
- [16] G. Metcalfe, *Fundamentals of fuzzy logic*, <https://api.semanticscholar.org/CorpusID:17549644>, 2005.
- [17] S. Owens, J. Reppy, and A. Turon, “Regular-expression derivatives re-examined,” *Journal of Functional Programming*, vol. 19, no. 2, pp. 173–190, 2009. DOI: 10.1017/S0956796808007090
- [18] A. Salomaa, “Two complete axiom systems for the algebra of regular events,” *Journal of the ACM*, vol. 13, no. 1, pp. 158–169, 1966. DOI: 10.1145/321312.321326
- [19] M. I. Sessa, “Approximate reasoning by similarity-based sld resolution,” *Theoretical Computer Science*, vol. 275, no. 1–2, pp. 389–426, 2002. DOI: 10.1016/S0304-3975(01)00188-8
- [20] A. Silva, F. Bonchi, M. Bonsangue, and J. Rutten, “Quantitative kleene coalgebras,” *Information and Computation*, vol. 209, no. 5, pp. 822–849, 2011. DOI: 10.1016/j.ic.2010.09.007
- [21] A. Stamenković and M. Ćirić, “Construction of fuzzy automata from fuzzy regular expressions,” *Fuzzy Sets and Systems*, vol. 199, pp. 1–27, 2012. DOI: 10.1016/j.fss.2012.01.007
- [22] The Rocq Community, *Regexp-Brzozowski*, <https://github.com/rocq-community/regexp-brzozowski>, 2024.
- [23] The Rocq Prover Development Team, *The rocq prover, version 9.1.0*, <https://rocq-prover.org/releases/9.1.0>, 2025.
- [24] K. Thompson, “Programming techniques: Regular expression search algorithm,” *Communications of the ACM*, vol. 11, no. 6, pp. 419–422, Jun. 1968, ISSN: 0001-0782. DOI: 10.1145/363347.363387

Appendices

Appendix A: Index of the Rocq Formalisation

This appendix indexes every Rocq definition, function, record and theorem that appears in a code listing of this thesis. The entries are grouped by topic, following the order in which they are developed in

Section 3. The source file of each group is named in its heading. The full development is available in the repository [10].

Rocq name	What it is	Page
Alphabet, words, and languages (Alphabet.v, Languages.v)		
SYM	Module type for an abstract alphabet of symbols	5
word	A word as a finite sequence of symbols	5
language	A language as a decidable predicate on words	5
void, eps, atom	The empty language, the language $\{\varepsilon\}$, and the singleton $\{a\}$	7
plus, conc, star	Union, concatenation, and Kleene star of languages	7
prod, compl	Intersection and complement of languages	7
Regular expressions and their semantics (RegexSemantics.v)		
regex	Inductive datatype of regular expressions (eight constructors)	6
restricted_regex	Predicate selecting the restricted fragment (no And/Neg)	6
den	Semantic interpretation $\mathcal{L}(\cdot)$ of an expression as a language	6
lang_eq	Extensional equality of two languages	14
re_equiv	Semantic equivalence of expressions ($\mathcal{L}(r) = \mathcal{L}(s)$)	14
Nullability (Nullable.v)		
has_eps	Boolean test for whether ε is in the language of an expression	9
nu	The nullability constant $\nu(r)$, either ε or $\emptyset$	9
nullable_correct	Lemma: has_eps agrees with semantic nullability	9
Canonicalisation (Canonicalization.v, CanonicalizationCorrectness.v)		
mkStar (smart constructors)	Smart constructors that build expressions in normal form	15
canonize / nf	The canonicalisation (normal-form) function on expressions	15
similar	Boolean similarity: equal normal forms	15
similar_iff_nf_eq	Theorem: $r \sim s$ iff $\text{nf}(r) = \text{nf}(s)$	16
canonize_correct	Theorem: normalisation preserves the denoted language	17
similar_sound	Theorem: similar expressions are semantically equivalent	17
Derivatives (Derivatives.v)		
dlang_char, dlang	Semantic (language) derivatives by a symbol and by a word	18
D_char	Syntactic derivative of an expression by a symbol	19
D	Syntactic derivative of an expression by a word	19
D_char_correct	Lemma: the symbol derivative matches the semantic one	20
D_correct	Theorem: the word derivative matches the semantic one	20
regular	Predicate: a language is denoted by some expression	21
derivative_regular	Theorem: derivatives of a regular language stay regular	21

continued on next page

Rocq name	What it is	Page
The derivative automaton (<code>Automata.v</code> , <code>AutomataConstruction.v</code>)		
<code>dfa</code>	Record type of a deterministic finite automaton	23
<code>constructed_dfa</code>	The (abstract) derivative DFA of an expression	23
<code>goto</code>	Recursive state-exploration building reachable states and transitions	23
<code>construct_dfa</code>	Executable driver constructing the derivative DFA	24
<code>delta_star</code>	Extended transition function: running the automaton on a word	26
<code>dfa_correctness</code>	Theorem: the constructed DFA accepts exactly $\mathcal{L}(r)$	26
T-norms and fuzzy relations (<code>SimilarityRelations.v</code>)		
<code>simval, in01</code>	Similarity values and the membership predicate for $[0, 1]$	28
<code>T_norm</code>	Record bundling a T-norm with its algebraic laws	28
<code>smin, godel_t_norm</code>	Gödel's minimum and its packaging as a T-norm	28
<code>cut_value, mu_cut</code>	A cut threshold μ and the μ -cut of a fuzzy relation	30
<code>fuzzy_rel, is_fuzzy_rel</code>	A fuzzy relation and the constraint that it lands in $[0, 1]$	30
<code>is_proximity</code>	Record: a reflexive, symmetric fuzzy relation	30
<code>is_T_similarity</code>	Record: a proximity satisfying the reverse triangle inequality	30
Induced similarities and finiteness (<code>SimilarityRelations.v</code>)		
<code>Rw_T</code>	Similarity on words induced by the symbol similarity	31
<code>RL_T</code>	Similarity on languages (infimum of suprema, both directions)	32
<code>GE</code>	Gödel similarity on regular expressions	32
<code>RE_T</code>	Syntactic similarity on expressions for a general T-norm	34
<code>general_similarity_relations</code>	Lemma: $\mathcal{R}^e, \mathcal{R}^w, \mathcal{R}^L$ are T -similarities	35
<code>syntactic_similarity_candidates</code>	Finite list of expressions sharing the shape of a given one	36
<code>finite_syntactic_similarity</code>	Lemma: only finitely many expressions are similar to a given one	36